Reinforcement Learning for Cybersecurity: A Flow-Based Defender for Binary Permit/Block Decisions

Javier Rivero Iglesias

May 2026

Contents

Chapter 1

Introduction

1.1 Motivation

Modern network infrastructures generate traffic volumes that far exceed what human analysts can manually inspect. A single enterprise network can produce millions of flow records per day, and large-scale deployments aggregate flows from thousands of endpoints, cloud services, and inter-datacenter links (Scarfone et al. 2007). Distinguishing legitimate communication from malicious activity within this volume is one of the core operational challenges in network security. Traditional responses rely on Network Intrusion Detection Systems that apply signature matching, statistical thresholds, or protocol-specific rules to flag suspicious activity. These approaches are effective against known threat patterns, but they require constant manual rule updates, are largely ineffective against novel attack variants, and generate alert volumes that routinely overwhelm security teams.

Machine learning offers a complementary perspective. Rather than encoding expert knowledge as static rules, ML models can learn statistical patterns from labeled traffic data and generalize toward previously unseen attack instances. As the field has matured, flow-level representations have become a practical substrate for this learning: each network connection is summarized as

a fixed set of statistical features covering duration, packet counts, byte totals, inter-arrival times, TCP flag distributions, and rate-based metrics (Sperotto et al. 2010; Umer et al. 2017). This reduction makes large-scale learning tractable and avoids the legal and technical complications of deep packet inspection, which is increasingly ineffective anyway as the fraction of encrypted traffic on the internet continues to grow.

The key limitation of standard supervised learning in this context is that it treats intrusion detection as a symmetric classification problem, minimizing overall misclassification error without accounting for the operational consequences of different error types (Axelsson 2000). In practice, those consequences are deeply asymmetric. A false negative permits a threat actor to proceed undetected and can lead to data exfiltration, ransomware deployment, or service disruption. A false positive generates an alert or traffic block that consumes analyst time, but rarely causes lasting damage. Reinforcement learning provides a natural framework for encoding this asymmetry directly into the learning objective: by designing a reward function that penalizes missed attacks more heavily than false alarms, an RL agent can be guided toward policies that reflect real operational risk priorities rather than aggregate accuracy (Sutton et al. 2018; Lee et al. 2002). Exploring how a distributional RL algorithm such as QRDQN navigates these trade-offs on flow-level data is the central motivation for this thesis (Dabney et al. 2018).

1.2 Problem Statement

Applying reinforcement learning to network intrusion detection raises several methodological challenges that go beyond substituting an RL algorithm for a standard classifier. The first challenge is data quality. Public NIDS benchmarks such as CICIDS2017 are widely used but contain known artifacts from their flow extraction pipeline, including label noise, duplicate features, CI-

CFlowMeter implementation bugs, and statistical fields that inadvertently encode capture-day context (Ring et al. 2019; Engelen et al. 2021; Lanvin et al. 2023). Models trained without auditing these artifacts may learn to recognize extraction signatures or scenario-specific identifiers rather than genuine attack behavior, producing inflated benchmark scores that do not transfer to any new environment.

The second challenge is data leakage. Many published NIDS studies evaluate models using random row-wise splits across all available traffic, which can place highly correlated flows on both sides of the train-test boundary. Beyond duplicate rows, leakage can enter through IP addresses, absolute timestamps, Flow IDs, and port numbers that are specific to a scripted attack scenario rather than indicative of malicious behavior more generally (Arp et al. 2020; Kaufman et al. 2011). Preprocessing steps such as feature scaling, applied globally before splitting, can further contaminate evaluation by allowing distributional information from the test partition to influence training normalization. A rigorous study cannot simply choose a high-performing algorithm and report its accuracy on a randomly split public dataset.

The third challenge is fair baseline comparison. Deep RL methods can be expensive to tune, sensitive to hyperparameters, and slower to converge than tabular supervised models. Random Forest and gradient-boosted tree ensembles are strong baselines for flow-level intrusion detection and can achieve near-perfect scores on CICIDS2017 under favorable evaluation conditions (H. Liu et al. 2019; Apruzzese et al. 2022). An RL-based defender cannot be meaningfully evaluated without a same-protocol comparison against such baselines. The question is not whether QRDQN achieves high accuracy on a single run, but whether its cost-sensitive formulation offers any empirical advantage over a well-tuned supervised model under matched feature schemas, split protocols, and evaluation metrics.

1.3 Proposed Approach

This thesis formulates the defensive task as an offline, dataset-as-environment reinforcement learning problem (Levine et al. 2020). The offline setting is deliberate: deploying an RL agent that must interact with a live network in order to learn would generate unacceptable security risks, since an exploring agent would inevitably take suboptimal actions during training. By treating a static labeled dataset as the environment, the agent can be trained and evaluated without any live network exposure. Network traffic is processed into a fixed, 76-feature canonical flow schema that is independent of any specific dataset's native format. This canonical contract separates extraction-specific column naming from the model's observation space and makes it possible to process both CICIDS2017 flows and independently captured laboratory traffic through the same pipeline without modification.

To handle missing or heterogeneous data when transitioning between datasets and capture tools, a 76-value missingness mask is appended to the feature vector, producing a 152-dimensional observation for the agent. Each position in the mask records whether the corresponding canonical feature was present and valid (encoded as 1) or was absent and replaced with a default value (encoded as 0). This makes data gaps visible to both the model and the evaluator, avoiding the silent imputation problem where a network learns on placeholder values without any indication that the underlying measurement was unavailable. When moving from CICIDS2017 to laboratory-captured traffic, different extraction pipelines may produce different column sets, and the missingness mask ensures that any such discrepancy is encoded in the observation rather than hidden from inspection.

The decision task is mapped to a binary action space: `PERMIT` for traffic classified as benign, and `BLOCK` for traffic classified as an attack. A Quantile Regression Deep Q-Network agent (Dabney et al. 2018), implemented through the

`sb3-contrib` library (Stable-Baselines3 Contributors 2023), is trained within a Gymnasium-compatible environment (Farama Foundation 2023). The environment applies an asymmetric reward function that penalizes false negatives more severely than false positives, directly encoding the operational risk priorities described above (Cárdenas et al. 2006). The CICIDS2017 dataset (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017) serves as the primary internal benchmark, processed through a curated loading pipeline that excludes leakage-prone fields before any model training begins. Performance is assessed against a Random Forest baseline under the same feature schema and split protocol (Engelen et al. 2021). A second validation tier extends the evaluation by running offline inference on flow features extracted from independently captured laboratory traffic, testing the system's resilience to domain shift.

1.4 Contribution of the Thesis

The primary contribution of this work is a reproducible, leakage-aware, and cost-sensitive experimental prototype for evaluating RL-based binary security decisions on network flow records. The system is built around an explicit canonical feature contract that separates dataset extraction artifacts from the model's observation space, making it possible to audit precisely which features are used, which are excluded, and why. This design choice directly addresses a gap in existing RL-based NIDS work, where feature selection decisions are often implicit and difficult to reproduce across different dataset formats or capture tools.

The second contribution is a multi-stage validation ladder that provides structured evidence about how model performance changes as evaluation conditions become progressively stricter. The ladder spans random stratified splits, day-pattern temporal splits, leave-one-exact-CSV-out evaluations, and

offline inference on independently captured laboratory traffic (Lanvin et al. 2023). Each rung tests a different failure mode: random splits test learning capacity, temporal splits test time-based generalization, CSV-out splits test sensitivity to specific capture scenarios, and lab-traffic inference tests resilience to domain shift. Reporting results across all rungs provides a more complete picture of model behavior than any single evaluation setting can offer.

The third contribution is a same-protocol empirical comparison between the QRDQN agent and a Random Forest baseline. Both models are trained on the same canonical features, evaluated under the same split protocol, and assessed using the same suite of metrics including accuracy, precision, recall, F1 score, false positive rate, and false negative rate. This comparison is designed to produce honest evidence about whether the RL formulation offers practical advantages, without overclaiming deployment readiness or generalizing beyond the controlled experimental conditions of the study.

1.5 Scope and Limitations

The scope of this research is strictly confined to offline, tabular flow-level evaluation. The `PERMIT` and `BLOCK` outputs are experimental binary classifications over static flow records; the system does not perform live firewall blocking, packet dropping, `iptables` manipulation, or inline intrusion prevention. The environment does not simulate an active adversary whose subsequent actions depend on the defender's choices, which means the system does not operate as a full sequential decision-making control plane. These constraints are deliberate design choices that make the evaluation tractable, reproducible, and safe. They are not omissions to be remedied in future iterations of this same prototype.

CICIDS2017 is used as the primary internal benchmark for controlled experimentation. Results on this dataset should be interpreted as evidence about

behavior under a specific, well-characterized benchmark, not as proof of generalization to arbitrary production networks. Independently captured laboratory traffic is used as a secondary evaluation tier to test domain shift, but this too is offline inference rather than live deployment. The thesis occupies a methodological position between pure benchmark evaluation and full deployment validation, targeting the reproducibility and interpretability of the experimental framework rather than operational deployability.

1.6 Structure of the Document

The remainder of this document is organized as follows. Chapter 2 details the specific objectives, scope boundaries, non-goals, and expected outcomes of the thesis. Chapter 3 reviews the state of the art, covering flow-based NIDS, public datasets including CICIDS2017, supervised learning baselines, reinforcement learning foundations, prior RL-based intrusion detection work, and common methodological pitfalls in ML security research. Chapter 4 outlines the methodological design, covering the data pipeline, the canonical feature schema, and the reinforcement learning formulation. Subsequent chapters cover the experimental setup and the final empirical results.

Chapter 2

Objectives and Scope

2.1 General Objective

The general objective of this thesis is to design, implement, and rigorously evaluate an offline, reinforcement learning-based cybersecurity defender that makes cost-sensitive binary decisions (`PERMIT` or `BLOCK`) over standardized network flow observations. This goal is pursued through a controlled experimental pipeline that operates entirely on static labeled flow data, prioritizing methodological rigor and reproducibility over claims of deployment readiness.

The approach builds on the observation that intrusion detection involves inherently asymmetric costs: permitting an attack carries consequences that are generally far more severe than those of generating a false alarm. A reward-driven formulation makes this asymmetry explicit in the learning objective rather than leaving it implicit in post-hoc threshold tuning. The central question guiding the work is whether a distributional RL algorithm such as QRDQN can be configured, trained, and evaluated in a way that demonstrates measurable cost-sensitive behavior on a well-studied public benchmark, while remaining comparable to strong supervised baselines under the same controlled conditions.

2.2 Specific Objectives

2.2.1 Flow-Based Traffic Representation

The first specific objective is to extract and normalize network traffic data into tabular flow records, ensuring that the input format is compatible with both public benchmarks such as CICIDS2017 and offline laboratory captures. Network flows are bidirectional summaries of packet exchanges sharing a common five-tuple, and their compact, fixed-width tabular structure makes them well-suited as inputs to machine learning models (Sperotto et al. 2010; Umer et al. 2017). Achieving this requires auditing the features available from CICFlowMeter-generated CSV files, identifying which fields carry legitimate statistical signal and which represent extraction artifacts or potential leakage vectors.

This objective also involves defining a stable feature contract that bridges the gap between academic CSV releases and raw CICFlowMeter exports. A stable contract means that the set of features used as model inputs does not change between experiments, that fields excluded for leakage reasons are excluded consistently, and that anyone reading the codebase can verify exactly which measurements reach the model. This stability is a prerequisite for all subsequent objectives, since every other component of the pipeline depends on a well-defined and auditable observation space.

2.2.2 Canonical Preprocessing and Feature Schema

The second specific objective is to define and enforce a strict 76-feature canonical schema alongside a 76-value missingness mask, yielding a stable 152-dimensional observation vector. The 76 canonical features are drawn from CICFlowMeter-style statistics covering flow duration, packet and byte counts,

inter-arrival time distributions, TCP flag counts, packet length statistics, active and idle period summaries, and flow rate metrics. Fields that could expose dataset-specific identity information, such as source and destination IP addresses, absolute timestamps, Flow IDs, and port numbers that function as direct attack-type proxies, are explicitly excluded at the schema level rather than filtered during ad hoc preprocessing steps.

The 76-value missingness mask records whether each canonical value was directly measured and present (encoded as 1) or absent and replaced with a default placeholder (encoded as 0). This design makes data gaps visible to the model and to the evaluator, preventing a situation where the network silently trains on imputed zeros with no indication that the underlying measurement was unavailable. When moving from CICIDS2017 to laboratory-captured traffic, different extraction pipelines may produce different column sets; the missingness mask ensures that any such discrepancy is encoded in the observation rather than hidden, so that a reader can distinguish a measured zero from an imputed one. Together, the canonical schema and the mask form the observation contract on which every downstream component depends.

2.2.3 Reinforcement Learning Environment

The third specific objective is to construct a Gymnasium-compatible dataset-as-environment interface (Farama Foundation 2023) that presents flow records to the agent as a stream of observations. The environment wraps a static labeled dataset so that each `step()` call returns one flow record as an observation, together with the reward computed from the agent's previous action and the true label of the last record. This formulation allows standard RL training loops, developed for interactive environments, to be applied directly to offline flow datasets without changes to the algorithm itself.

The environment must implement an asymmetric, cost-sensitive reward function that penalizes false negatives more heavily than false positives, re-

flecting cybersecurity risk profiles in which missed attacks are operationally more damaging than false alarms (Lee et al. 2002; Cárdenas et al. 2006). The specific reward weights are treated as configurable experimental parameters rather than fixed universal constants, since the appropriate cost ratio depends on the threat model and operational context of the deployment scenario. The environment must also support reproducible resets, episode boundary management, and configurable dataset splits so that the same interface can be used for both training and evaluation under different split modes.

2.2.4 QRDQN-Based Binary Decision Agent

The fourth specific objective is to configure and train a Quantile Regression Deep Q-Network agent (Dabney et al. 2018), using its distributional approach to value estimation, to navigate the binary `PERMIT/BLOCK` action space based on the 152-dimensional canonical observations. QRDQN extends standard DQN by estimating the full distribution of expected returns rather than only the mean, using a set of learned quantiles and a quantile regression loss. This distributional representation is of interest in a cost-sensitive setting because it provides richer information about the spread and shape of possible outcomes, not just the average return, and in principle allows the policy to be made more conservative about worst-case results.

The practical implementation uses the `sb3-contrib` library's QRDQN module (Stable-Baselines3 Contributors 2023), which provides a stable, tested implementation of the algorithm with configurable network architecture, replay buffer, exploration schedule, and training hyperparameters. The objective includes selecting a sensible default configuration, validating that the training loop converges on the internal benchmark, and persisting all trained model artifacts alongside the configuration and preprocessing state used to produce them. Reproducibility requires that a stored model can be loaded and evaluated identically at a later date without reconstructing the full training pipeline

from scratch.

2.2.5 Supervised Baseline Comparison

The fifth specific objective is to implement a robust supervised baseline, primarily using Random Forest (H. Liu et al. 2019), to provide a fair, same-protocol comparative assessment of the QRDQN agent's empirical value. Random Forest is chosen as the primary baseline because it is well-established for tabular classification, handles non-linear feature interactions without extensive preprocessing, is robust to irrelevant features, and tends to perform strongly on flow-based NIDS benchmarks (Apruzzese et al. 2022). Using the same feature schema, the same split protocol, and the same evaluation metrics for both the RL agent and the baseline is essential; any discrepancy in preprocessing, feature availability, or evaluation conditions would undermine the validity of the comparison.

This comparison serves several purposes. First, it determines whether the QRDQN formulation offers any empirical advantage over a well-tuned supervised classifier under controlled conditions. Second, it provides a sanity check on the difficulty of the benchmark task, since a Random Forest achieving very high scores contextualizes whatever QRDQN achieves. Third, it grounds any discussion of the cost-sensitive reward design in concrete evidence. If both models are evaluated using the same cost-weighted metrics, the comparison reveals whether the RL reward function actually drives the agent toward a different operating point than a supervised model trained on the same data, or whether the two approaches converge to similar behavior.

2.2.6 Leakage-Aware Evaluation

The sixth specific objective is to design preprocessing pipelines and validation protocols that strictly prevent data leakage throughout all evaluation tiers.

This requires the programmatic exclusion of identifying fields, such as source and destination IP addresses, absolute timestamps, Flow IDs, and direct port proxies, at the data loading stage before any model training begins (Arp et al. 2020; Kaufman et al. 2011). These fields are excluded at the schema level through the canonical feature contract rather than relying on ad hoc filtering that could be inconsistently applied across different runs or dataset versions.

Scaling transforms must be fitted exclusively on the training partition and then applied to validation and test partitions using the parameters learned from training data alone. Fitting a scaler on the full dataset before splitting allows distributional information from the test set to influence the normalization of training features, a subtle form of leakage that can inflate reported performance on datasets with strong temporal or scenario structure. The fitted scaler is persisted as part of the run artifact so that it can be reloaded and applied identically during Phase 2 lab-traffic inference, ensuring that the same normalization is used for out-of-distribution flows without any information about their distribution reaching the scaler state.

2.2.7 Training-Scale and Data-Efficiency Analysis

The seventh specific objective is to evaluate the performance of both the RL agent and the supervised baselines under varying amounts of training data, identifying the data-efficiency characteristics of the proposed formulation and assessing how rapidly QRDQN converges compared to supervised alternatives. Deep RL methods can be sample-hungry, and their learning dynamics differ substantially from those of ensemble tree methods. Understanding how much labeled traffic is required for each approach to reach useful performance is practically relevant: labeled attack traffic is expensive to collect, and real deployment scenarios may not have access to the large, balanced datasets available in academic benchmarks (Ring et al. 2019).

Training-scale experiments use progressively larger training subsets drawn

from the same split, measuring how precision, recall, F1, false positive rate, and false negative rate change as the budget grows. The comparison between QRDQN and Random Forest at each budget level reveals whether the RL formulation has a higher sample requirement, whether it converges more slowly, or whether it reaches a different final performance plateau. These results are interpreted as internal evidence about this specific formulation and benchmark combination, not as general claims about RL data efficiency in all NIDS settings.

2.2.8 External Validation with Laboratory Traffic

The eighth specific objective is to construct a robust offline inference pipeline capable of ingesting independently captured traffic from a private laboratory setup and producing predictions from a model trained on CICIDS2017. This validation tier tests whether the canonical feature schema and the trained model generalize beyond the benchmark distribution to flows extracted from a different network environment, potentially using a different version of CICFlowMeter or a different capture configuration. Domain shift between the training distribution and the inference distribution is a fundamental challenge for NIDS models and is rarely tested in studies that evaluate only on a single public dataset (Apruzzese et al. 2022; Layeghy et al. 2023).

The inference pipeline applies optional percentile-based and z-score clipping to handle out-of-distribution raw feature values that fall outside the range seen during CICIDS2017 training. The fitted scaler from Phase 1 is loaded and applied without refitting, preserving the integrity of the train-test separation. The missingness mask in the observation vector explicitly encodes any features that could not be computed from the laboratory capture, making the incomplete-data case visible in the prediction input rather than silently imputed. The expected output of this objective is a set of prediction artifacts and metrics that allow the laboratory traffic results to be directly compared

to the Phase 1 validation results, providing concrete evidence about how performance changes under domain shift.

2.3 Scope of the Work

The work is bounded by the requirements of an offline evaluation of a tabular machine learning pipeline. It encompasses the ingestion of CSV flow records from both CICIDS2017 and laboratory captures, the application of standard scaling and optional outlier clipping, the execution of training loops through the Stable Baselines3 framework (`sb3-contrib`), and the generation of structured, reproducible run artifacts that persist models, metrics, scalers, and configuration files alongside each training run. The internal experimentation is conducted entirely on the CICIDS2017 dataset, using explicit train-test splits and a multi-stage validation ladder to evaluate model behavior under progressively stricter conditions before testing on externally captured traffic.

The scope explicitly includes the multi-stage validation ladder described in the evaluation objectives. This ladder covers random stratified splits for baseline learning checks, a shuffled-label sanity check to verify that performance collapses when labels carry no real signal, a day-pattern or CSV-based temporal split to test generalization within CICIDS2017, a leave-one-exact-CSV-out evaluation to test sensitivity to specific capture files, and offline inference on independently captured laboratory flows. Each tier contributes distinct evidence and is part of the agreed experimental scope.

2.4 Non-Goals

This thesis explicitly excludes the development of inline network monitoring or live packet interception. The system will not integrate with firewalls, `iptables`

rules, Software-Defined Networking (SDN) controllers, or active Intrusion Prevention Systems (IPS). These exclusions exist because live deployment would require a substantially different system design, safety validation, and operational testing methodology that falls outside the scope of a controlled academic study. The thesis is designed as an experimental prototype, not as an operational product intended for production use.

The thesis also does not attempt to solve the broader problem of multi-agent adversarial cyber operations or sequence-dependent attack disruption, where the environment state changes dynamically in response to the defender's actions. In the current formulation, each flow record is classified independently, and the dataset does not react to the agent's decisions. This is a known limitation of the dataset-as-environment approach, and it means the system cannot model adversarial adaptation, attacker persistence, or multi-stage attack chains that span many individual flow decisions. Finally, the goal is not to demonstrate that QRDQN represents the definitive state of the art for all NIDS tasks, but rather to evaluate its behavior under a carefully controlled, leakage-free, and reproducible experimental framework and compare it honestly with supervised baselines.

2.5 Expected Outcomes

The expected outcome of this thesis is a fully functional, documented, and reproducible offline inference pipeline supported by a detailed empirical analysis. The project will yield persistent artifacts for all training and validation runs, covering trained model files, fitted scalers, metric reports, validation outputs, and configuration records that fully specify the conditions under which each result was produced. This artifact discipline is part of the methodological contribution: a result that cannot be reproduced from its artifact trail provides weaker evidence than one that can.

In terms of empirical content, the project will produce precision, recall, F1 score, false positive rate, and false negative rate measurements for both the QRDQN agent and the Random Forest baseline across all validation tiers. The training-scale experiments will produce learning curves that characterize how each model's performance evolves as the training budget grows. The Phase 2 inference results will characterize how performance changes when the model encounters independently captured laboratory traffic rather than held-out CIC-IDS2017 flows. Taken together, these results will quantify the generalization limits of benchmark-trained RL models when subjected to strict temporal splits, leave-one-CSV-out evaluation, and independent laboratory traffic, and will provide an honest empirical assessment of whether the cost-sensitive RL formulation offers measurable advantages over a well-matched supervised baseline.

Chapter 3

State of the Art

3.1 Introduction and Scope of the Chapter

This chapter maps the technical background for a reinforcement-learning-based cybersecurity defender that operates on flow-level network observations. The scope is deliberately broader than a comparison of binary classifiers. The thesis combines network intrusion detection, flow-feature representation, public benchmark datasets, supervised machine learning baselines, a Gymnasium-style dataset-as-environment formulation, QRDQN, leakage-aware preprocessing, stricter validation checks, data-efficiency experiments, and planned or preferred validation on independently captured laboratory traffic.

The chapter therefore follows a funnel. It first reviews network intrusion detection and traffic representation, then situates CICIDS2017 among public datasets, then discusses supervised and deep learning baselines, and finally places the RL formulation within both reinforcement-learning theory and prior RL/DRL work for intrusion detection and cyber defense. The final sections focus on methodological risks, evaluation protocols, training-scale evidence, and the research gap addressed by this thesis.

The central claim is cautious. Reinforcement learning for intrusion detection already exists, and public NIDS benchmarks are widely used (Yang et al.

2026; Kheddar et al. 2025; Gueriani et al. 2024). This thesis does not claim to introduce the first RL IDS, to prove QRDQN as the NIDS state of the art, or to demonstrate production-ready inline prevention. It studies a reproducible, offline, flow-level `PERMIT/BLOCK` decision pipeline and evaluates how that formulation behaves under controlled internal benchmarks, baseline comparison, and progressively stricter validation.

The review also separates three levels that are often conflated. The first level is monitoring: collecting packets, flows, logs, or derived events and deciding whether they are suspicious. The second is prediction: training a model to map traffic representations to benign or malicious labels. The third is control: taking actions that change the future network state. This thesis works mainly at the first two levels. It uses the language of `PERMIT` and `BLOCK` because the environment exposes a binary security decision, but the current implementation is not an inline control plane. That distinction shapes every later claim.

3.2 Intrusion Detection and Network Security Monitoring

Intrusion Detection Systems monitor host or network activity for evidence of attacks, misuse, or policy violations (Scarfone et al. 2007). A Host Intrusion Detection System observes endpoints, operating-system events, logs, processes, file-integrity changes, or system calls. A Network Intrusion Detection System observes traffic crossing links, taps, mirrored switch ports, packet brokers, virtual-network interfaces, or flow collectors. The two perspectives are complementary: HIDS can see local process and file context that network sensors cannot, while NIDS can inspect lateral movement, scans, command-and-control traffic, and aggregate communication patterns without installing an agent on every host.

IDS must also be distinguished from Intrusion Prevention Systems. An IDS primarily detects and reports suspicious activity. Its output may be an alert, log event, enriched flow record, ticket, SIEM event, or forensic artifact. An IPS is positioned inline and can block, drop, modify, rate-limit, reset, or redirect traffic (Scarfone et al. 2007). In practice, products and architectures often blur this boundary because a detection engine may feed firewall rules, EDR actions, SOAR playbooks, or SDN policy updates. For evaluation, however, the distinction matters. A detector can tolerate a different false-positive profile from an inline blocker, because false alerts mainly affect analyst workload whereas false blocks can interrupt legitimate business traffic.

Classical IDS taxonomies distinguish signature-based, anomaly-based, specification-based, and hybrid approaches (Scarfone et al. 2007). Signature-based detection matches known malicious patterns: byte sequences, rule predicates, protocol conditions, command strings, exploit traces, or known behavior combinations. Its strength is precision and explainability for known threats. Its weakness is dependency on rule maintenance and poor coverage of novel attacks, polymorphic behavior, encrypted payloads, and environment-specific variations.

Anomaly-based detection models expected behavior and flags deviations (Sommer et al. 2010; Ring et al. 2019). This is the family most naturally associated with ML-based NIDS, but the word anomaly should not be interpreted as automatically meaning unsupervised learning. Many public NIDS studies are supervised classifiers trained on labeled benign and attack flows, even when the broader motivation is anomaly detection. The operational challenge is that benign traffic is broad and non-stationary, attack prevalence is low, labels are incomplete, and changes in business activity can look anomalous without being malicious.

Specification-based detection encodes allowed or expected behavior, for example protocol state machines, allowed command sequences, or restricted industrial-control workflows. It can be powerful when the monitored system

has narrow, stable behavior, but it requires domain knowledge and maintenance. Hybrid systems combine these ideas, such as signature rules for known exploits, statistical anomaly scores for unusual flow behavior, and host telemetry for endpoint confirmation. Modern monitoring programs rarely depend on one detection paradigm alone.

Alerting and active response form a separate axis. A system may only produce alerts for later triage, may enrich alerts with contextual evidence, may recommend response steps, or may trigger active containment. This thesis is intentionally on the conservative side of that axis. It studies an offline NIDS-like decision model over extracted flow rows. The model emits `PERMIT/BLOCK` predictions during experiments, but it does not drop packets, rewrite firewall policies, update SDN rules, quarantine hosts, or operate in a live feedback loop. The NIDS literature is therefore relevant, but production IPS claims are outside the supported scope.

3.3 Network Traffic Representation

Network traffic can be represented as packets, payload bytes, sessions, connections, or flows. Packet-level representations preserve per-packet timing, headers, sizes, directions, flags, and sometimes partial payload. They support fine-grained protocol analysis and sequence modeling, but they are expensive to store and process on high-speed networks. Payload-level representations retain application content and can identify exploits or malware signatures that are invisible in metadata. Their usefulness is limited by encryption, privacy constraints, legal restrictions, and CPU cost.

Session- or connection-level representations group packets belonging to the same exchange, often using the five-tuple of source IP, destination IP, source port, destination port, and protocol. They may include connection state, handshake behavior, application metadata, or TLS/HTTP-derived attributes.

Flow-level representations are more compact: they aggregate packets over a time window or connection-like record and summarize behavior through duration, bytes, packets, directionality, inter-arrival times, packet-length statistics, TCP flag counts, and active or idle periods (Sperotto et al. 2010; Umer et al. 2017).

NetFlow and IPFIX-style records are the operational reference point for scalable flow monitoring. Routers, switches, probes, and collectors export compact flow records that can be retained for long periods and processed at scale. These records are attractive for NIDS because they reduce volume and avoid payload inspection. They are also limited because deployments often export only a subset of possible fields, may sample traffic, and may omit richer statistics used in academic datasets. This creates a gap between production flow telemetry and feature-rich benchmark CSVs.

CICFlowMeter-style features sit on the richer, research-oriented side of that gap. CICIDS2017 provides PCAPs and generated bidirectional flow CSVs, with features such as flow duration, forward and backward packet and byte counts, inter-arrival-time statistics, packet-length statistics, TCP flag counts, header-length aggregates, flow rates, and active/idle timing (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). These features are convenient for tabular ML because each flow becomes a fixed-width row, but they depend on offline PCAP reconstruction and feature-extraction choices.

The suitability of flow-level representation is therefore conditional. It is suitable for this thesis because the implemented defender consumes fixed numerical vectors, because CICIDS2017 is already released in flow form, and because the planned Phase 2 path also extracts flow CSVs before inference. It is limited because payload semantics, exact packet ordering, application content, and some multi-flow temporal dependencies are lost. Attacks whose evidence is mainly content-level, host-level, or long-horizon behavioral may be only indirectly visible to a flow classifier (Sperotto et al. 2010; Umer et al.

2017).

Feature availability is another limitation. A feature computed by CICFlowMeter from full PCAPs may not be available from a production NetFlow/IPFIX exporter, or may be computed differently. Sarhan et al. address this problem by mapping heterogeneous NIDS feature sets toward a standard NetFlow-aligned representation to improve generalisability and explainability (Sarhan et al. 2021). This thesis does not adopt Sarhan's exact standard feature set, but it follows the same methodological motivation: define a stable feature contract, document what is present or missing, and avoid treating a benchmark-specific CSV schema as if it were universal.

The current project uses a canonical flow schema with 76 feature values and a 76-value missingness mask, producing 152-dimensional observations. The mask records whether each canonical value was present and valid, rather than silently hiding missing or imputed fields. This matters when moving from CICIDS2017 to laboratory traffic because different extraction pipelines may not produce exactly the same columns. The mask does not make missing data harmless, and it does not solve domain shift. It makes the model input auditable: a reader can distinguish a measured zero from an imputed placeholder.

Flow representation also changes the privacy and leakage profile. Removing payloads can reduce privacy exposure, but metadata can still identify users, services, schedules, and scenarios. IP addresses, absolute timestamps, Flow IDs, endpoint identity, capture-day structure, and ports used as scenario proxies can leak labels rather than represent attack behavior (Arp et al. 2020; Kaufman et al. 2011). The thesis therefore treats representation and feature selection as security-relevant design choices, not only preprocessing details.

3.4 Public Datasets for Network Intrusion Detection

Public NIDS datasets enable shared experimentation, reproducibility, and comparison across papers (Ring et al. 2019; Thakkar et al. 2020). They also constrain conclusions. A dataset reflects its traffic generator, topology, capture process, feature extractor, labels, attack scripts, and release format. Performance on a public benchmark is therefore evidence about a controlled task, not direct evidence of deployment readiness (Apruzzese et al. 2022; Layeghy et al. 2023).

KDDCup99 and NSL-KDD are historically important because they shaped early IDS evaluation (KDD Cup 1999; Tavallaee et al. 2009). KDDCup99 contains connection records derived from DARPA-era traffic and uses a 41-feature schema with attack categories such as DoS, Probe, R2L, and U2R. NSL-KDD reduced some redundancy problems and made the benchmark easier to use, but it retained the same basic origin and feature logic. These datasets are useful for understanding the history of IDS evaluation, but they are weak evidence for modern flow-based defense because their traffic, attack behavior, and schema are old (Tavallaee et al. 2009; Ring et al. 2019). In this repository, NSL-KDD is therefore historical context, not the final path for the current defender.

UNSW-NB15 is a more recent cyber-range dataset generated with contemporary benign and malicious traffic, raw captures, and engineered features (Moustafa and Slay 2015; Moustafa and Slay 2016). It remains useful because it broadened the field beyond KDD-style benchmarks and includes a richer set of attack categories. CICIDS2017 and CSE-CIC-IDS2018 are widely used CIC-family datasets for flow-based NIDS research, both providing modern lab-generated attack scenarios and flow features derived from packet captures (Sharafaldin et al. 2018; Communications Security Establishment et al. 2018).

Bot-IoT and ToN_IoT broaden the landscape toward IoT and IIoT traffic, where device behavior, telemetry, and attack patterns differ from enterprise networks (Koroniotis et al. 2019; Moustafa, Ahmed, et al. 2020; Alsaedi et al. 2020).

Dataset	Benchmark role	Representation	Main strengths	Main caveats
KDDCup99	Legacy historical benchmark	Connection records with 41 handcrafted features	Very widely used; simple baseline for older IDS literature (KDD Cup 1999)	Outdated traffic; redundancy; weak proxy for modern NIDS (Ring et al. 2019)
NSL-KDD	Cleaned legacy benchmark	Same KDD-style connection schema	Reduces some duplicate-record issues; useful for historical comparison (Tavallaee et al. 2009)	Still inherits old DARPA/KDD assumptions and feature design
UNSW-NB15	Modern lab/cyber-range dataset	PCAPs, logs, and engineered tabular features	More contemporary attack taxonomy and richer features (Moustafa and Slay 2015)	Controlled environment; not direct production evidence
CICIDS2017	Main internal thesis benchmark	PCAPs and CICFlowMeter flow CSVs	Public PCAPs; multiple attack days; rich flow features (Sharafaldin et al. 2018)	Known label, extraction, duplication, and split-sensitivity concerns (Engelen et al. 2021; Lanvin et al. 2023)
CSE-CIC-IDS2018	Larger CIC-family benchmark	PCAPs and CICFlowMeter-style flows	Broader CIC/CSE collaboration; useful modern comparison (Communications Security Establishment et al. 2018)	Similar lab-generation and flow-artifact caveats (L. Liu et al. 2022)
Bot-IoT	IoT/botnet benchmark	PCAPs, Argus-style flows, CSV exports	IoT-focused botnet, scan, DoS/DDoS, and exfiltration scenarios (Koroniotis et al. 2019)	Strong imbalance and dataset-specific topology artifacts
ToN_IoT	IoT/IIoT telemetry benchmark	Network flows, host logs, and sensor telemetry	Multi-modal IoT/IIoT perspective (Alsaedi et al. 2020)	Niche environment; identifier and scenario artifacts remain possible

Table 3.1: Compact comparison of public NIDS datasets relevant to the thesis.

Table 3.1 is not a ranking. It shows why dataset choice must be aligned with claim strength. Historical benchmarks help situate the field; modern lab datasets support controlled comparison; IoT datasets test different traffic assumptions; external laboratory or production-like traffic is needed for stronger generalisation evidence. None of these datasets should be treated as a complete proxy for all real networks. Their value is highest when used with transparent

preprocessing, strong baselines, and validation protocols that test distribution shift rather than only random matched-distribution performance.

3.5 CICIDS2017 as the Main Internal Benchmark

CICIDS2017 is the main internal benchmark in this thesis because it provides labeled benign and attack traffic, raw PCAPs, and flow CSVs generated with CICFlowMeter-style features (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). It was designed to improve on older IDS datasets by using contemporary traffic profiles, realistic benign activity profiles, and multiple attack scenarios (Sharafaldin et al. 2018; Sharafaldin et al. 2019). These properties make it suitable for controlled experimentation with flow-level NIDS models.

Its structure also makes it attractive for thesis work. The dataset spans multiple capture days and attack families, and it is available both as raw packet captures and as generated CSV files. The PCAPs are closer to the original measurement evidence. The CSVs are easier to use for ML because CICFlowMeter has already reconstructed bidirectional flows and computed statistical features. This convenience is also a risk: once researchers work only with summarized CSVs, they may stop checking how the flows were generated, how labels were assigned, and whether rows preserve independence across train and test partitions.

CICFlowMeter reconstructs flows from packet traces using endpoint keys, directionality, timeout logic, and feature computations. Small differences in extraction settings can change the resulting rows, durations, counts, and labels. Engelen et al. show that CICIDS2017 contains practical issues in flow reconstruction and dataset consistency (Engelen et al. 2021). Lanvin et al.

further report errors that affect detection performance and show that apparently small dataset corrections can produce significant metric differences (Lanvin et al. 2023; Lanvin et al. 2022). Liu et al. analyze error prevalence in CIC-IDS-2017 and CSE-CIC-IDS-2018, while Rosay et al. provide a broader analysis and reconstruction-oriented discussion (L. Liu et al. 2022; Rosay et al. 2022). Dube's critique is especially relevant to claim discipline because it questions common interpretations of high scores on summarized CIC-IDS 2017 data (Dube 2024).

These critiques do not make CICIDS2017 useless. They make irresponsible use easier to identify. Common misuse patterns include treating the pregenerated CSVs as unquestioned ground truth, using random row-wise splits across all days, keeping identifiers or timestamps that expose scenario context, reporting only accuracy, ignoring class imbalance, and failing to document whether the data are original, corrected, reconstructed, or curated. Corrected or reconstructed variants can be valuable, but they are not interchangeable with the original CSV release. A thesis must name which variant it uses and avoid mixing results from different variants as if they came from one dataset.

The current repository responds to these concerns through a curated tracked dataset copy, a separate untracked raw reference copy, and loader-level cleaning in `src/load_cicids2017.py`. The adapter performs numeric coercion, invalid-value handling, canonical mapping, and anti-leakage exclusion of fields that should not be model features. The code also fits scaling on the training split and applies the fitted transform to the test split, rather than fitting on all data. The canonical mapping in `src/canonical_schema.py` excludes IPs, timestamps, Flow IDs, and specific ports from the canonical features.

The evaluation code also supports random splits, CSV/day-style splits, and leave-one-exact-CSV-out validation. Random split performance remains useful for checking whether the model and implementation can learn the internal benchmark. Harder day/file splits, shuffled-label checks, leave-one-exact-CSV-

out validation, and external lab-flow inference test different failure modes. A high score on the easiest setting is therefore not sufficient for a deployment claim. In this thesis, CICIDS2017 should be read as the main internal benchmark, not as a substitute for independent traffic validation.

3.6 Supervised Machine Learning and Deep Learning for NIDS

Most flow-based NIDS work is naturally framed as supervised learning: each traffic record is mapped to a benign, malicious, or attack-family label (H. Liu et al. 2019; Ahmad et al. 2021). Classical supervised methods include decision trees, Random Forests, SVMs, k-NN, logistic regression, Naive Bayes, and gradient-boosted ensembles. These models differ in assumptions and operational cost. Logistic regression is simple and interpretable but limited for non-linear feature interactions. Naive Bayes is fast and useful as a weak baseline, but its independence assumptions are unrealistic for correlated flow statistics. k-NN can capture local neighborhoods but scales poorly and is sensitive to feature scaling. SVMs can be strong on smaller datasets but become expensive on very large flow tables and require careful scaling and kernel selection.

Tree-based models remain especially important for tabular NIDS. Decision trees and Random Forests handle non-linear interactions, heterogeneous feature scales, irrelevant variables, and mixed feature distributions well (H. Liu et al. 2019; Apruzzese et al. 2022). Gradient-boosted decision-tree families such as XGBoost, LightGBM, and CatBoost are often strong tabular learners, but this chapter does not need separate claims about each implementation unless the thesis evaluates them directly. The important methodological point is simpler: an RL-based flow classifier must be compared against competent tabular supervised baselines, not only against weak or outdated methods.

This matters directly for the thesis. If each observation is a labeled flow row and the reward is derived from classification correctness, a supervised classifier is the natural baseline. A QRDQN agent cannot be evaluated only against its own reward curve, training loss, or a majority-class baseline. It must be compared with strong supervised methods under the same feature schema, split protocol, preprocessing, and metrics. The current repository includes a Random Forest baseline protocol aligned with the QRDQN splits, while the maintained results snapshot still marks the latest full metrics as pending. Until those artifacts exist, the chapter should describe the baseline obligation rather than claim a completed comparison.

Deep learning expands the design space through MLPs, CNNs, RNNs, LSTMs, autoencoders, attention mechanisms, transformer-style models, and graph-based approaches (Xiao et al. 2021; Asadullah et al. 2023; Sayeeduddin et al. 2022; Nguyen et al. 2022). MLPs treat flow features as generic numerical vectors. CNNs are used when packet bytes, feature matrices, or local feature patterns are encoded in a structured form. RNNs and LSTMs target temporal sequences of packets, flows, or events. Autoencoders often support anomaly detection by learning compressed representations of benign behavior and flagging reconstruction errors. Transformer-style models and graph neural networks are newer directions for sequence and structural traffic representations.

These architectures can report very high benchmark scores, especially on public datasets, but surveys also identify recurring weaknesses: unclear preprocessing, favorable random splits, missing class-imbalance analysis, limited external validation, and insufficient reproducibility (H. Liu et al. 2019; Ring et al. 2019; Arp et al. 2020). A deep model can overfit benchmark artifacts as easily as a shallow model. The model family alone does not solve leakage, class imbalance, feature availability, or domain shift.

Feature standardisation is another important supervised-learning lesson. Different datasets and tools expose overlapping but non-identical feature sets, which makes cross-dataset comparison difficult. Sarhan et al. proposed a

NetFlow-aligned feature mapping to improve generalisability and explainability across NIDS datasets (Sarhan et al. 2021). This thesis does not adopt that exact standard, but it adopts the same motivation: a fixed canonical schema makes the model interface auditable and allows CICIDS2017 and laboratory flow inputs to be compared through one preprocessing contract. The missingness mask extends this idea by making absent features visible to the model and to the evaluator.

3.7 Reinforcement Learning Foundations

Reinforcement learning studies how an agent selects actions in an environment to maximize return (Sutton et al. 2018). Its core concepts are states or observations, actions, rewards, policies, value functions, and transitions. In a Markov decision process, the current state summarizes the information needed for future decision-making, and actions influence both immediate reward and later states (Sutton et al. 2018). This sequential property is what distinguishes RL from ordinary supervised classification.

A policy specifies how the agent chooses actions from observations. A value function estimates expected return from a state, while an action-value function estimates expected return after taking a particular action in a state. Rewards define the task objective, and return aggregates future rewards, often with discounting. These definitions matter because changing the reward function changes what the agent is optimized to do. In a security setting, a reward that penalizes false negatives more heavily than false positives encodes one operational preference; it is not a neutral fact about all networks.

The episodic versus continuing distinction is also relevant. Episodic tasks have boundaries and terminal states, such as a game episode or a simulated incident-response scenario. Continuing tasks model ongoing operation, such as continuous network monitoring. A static dataset-as-environment formulation

often imposes artificial episodes: an episode may be a fixed number of sampled rows, a pass through a dataset, or a file. That is acceptable for experimentation, but it is weaker than an environment where the defender's action changes the next state.

Classical Q-learning is a model-free, off-policy temporal-difference method that learns action values without requiring a model of environment dynamics (Watkins et al. 1992). Model-free methods learn directly from interaction or logged transitions, while model-based methods learn or use a transition model for planning. On-policy methods learn about the policy that is currently generating behavior; off-policy methods can learn about a target policy while behavior is generated differently. DQN and QRDQN inherit the value-based, off-policy framing. Experience replay, introduced earlier as a mechanism for reusing past experience, also became central to later deep Q-learning systems (L.-J. Lin 1992).

Tabular RL is not appropriate when observations are high-dimensional numerical vectors. A lookup table over all possible flow-feature combinations is infeasible, and small numerical changes could create effectively new states. Function approximation is therefore necessary. Deep RL uses neural networks to approximate policies or value functions over large observation spaces. In this thesis, the observation is a 152-dimensional vector: 76 canonical flow values plus 76 missingness-mask values. That justifies a DQN-family algorithm more than a tabular RL algorithm, but it does not by itself justify RL over supervised learning.

Cybersecurity can contain genuine sequential RL problems. A defender may isolate a host, patch a service, change routing, deploy a countermeasure, or choose where to inspect next, and those actions can affect later observations and attacker opportunities. However, a static labeled-flow dataset is a weaker setting. If one flow row is sampled, classified, and followed by another row that is not causally affected by the previous action, the problem resembles cost-sensitive classification or a contextual decision process more than full au-

tonomous cyber defense (Levine et al. 2020; Fujimoto et al. 2019). The thesis must keep this distinction explicit.

3.8 Deep Q-Learning and Distributional Reinforcement Learning

Deep Q-Learning replaces a tabular action-value function with a neural network, making value-based RL applicable to high-dimensional observations (Mnih et al. 2015). DQN popularized two stabilizing mechanisms: replay memory and a separate target network (Mnih et al. 2015). Replay memory breaks short-term correlations by sampling past transitions for training and improves data reuse. A target network stabilizes bootstrapped targets by keeping the target calculation separate from rapidly changing online network weights.

Later DQN variants addressed known weaknesses. Double DQN reduces overestimation bias by decoupling action selection from target evaluation (Hasselt et al. 2016). Dueling DQN separates state value and action advantage estimation, which can help when many actions have similar values in a state (Wang et al. 2016). Prioritized Experience Replay samples transitions according to learning signal rather than uniformly (Schaul et al. 2016). Rainbow combined several improvements into a single DQN-family agent, including double Q-learning, dueling networks, prioritized replay, multi-step returns, distributional learning, and noisy exploration (Hessel et al. 2018).

Distributional reinforcement learning changes the object being estimated. Instead of only estimating expected return, it approximates the distribution of possible returns (Bellemare et al. 2017). C51 represents the return distribution over a fixed categorical support. QRDQN uses quantile regression to approximate return quantiles, avoiding the same fixed support assumption (Dabney

et al. 2018). QRDQN is therefore not a separate paradigm from DQN; it is a distributional, value-based member of the DQN family.

The security motivation is intuitive but must be stated carefully. False positives and false negatives have asymmetric consequences, and rare high-cost errors can matter more than a mean score suggests. A distributional value estimate may be useful for studying return structure under asymmetric rewards. It may expose whether a decision has a narrow or broad return distribution under the learned model. However, distributional RL does not automatically provide calibrated uncertainty, operational safety, risk-sensitive behavior, or superiority over scalar DQN. Those properties would require explicit decision rules, calibration, tail-risk objectives, or evaluation criteria beyond simply training a distributional agent (Bellemare et al. 2017; Dabney et al. 2018).

In this thesis, QRDQN is an exploratory algorithmic choice. It is suitable for a discrete binary action space and moderately high-dimensional flow observations, and it is available through the SB3-Contrib implementation used by the project (Stable-Baselines3 Contributors 2023; Farama Foundation 2023). The algorithmic choice is coherent: two actions, numerical observations, off-policy value learning, replay, and a distributional target. The evidential claim remains empirical. QRDQN should not be described as proven superior for NIDS unless same-protocol comparisons against DQN, Random Forest, and other baselines support that claim.

3.9 Reinforcement Learning for Intrusion Detection and Cyber Defense

RL and DRL for intrusion detection already form an active literature (Yang et al. 2026; Kheddar et al. 2025; Gueriani et al. 2024; Adawadkar et al. 2022). Prior work includes DQN-style intrusion classifiers, actor-critic approaches,

adversarial-environment formulations, feature-selection agents, SDN or IoT-focused systems, and autonomous cyber-defense simulations. The field is heterogeneous, so papers should not be treated as directly comparable unless their task formulation, datasets, actions, rewards, and validation protocols are aligned.

One branch is close to this thesis: dataset-as-environment intrusion detection. Lopez-Martin et al. reformulated supervised intrusion detection as a deep RL problem by treating labeled records as states, class predictions as actions, and reward as a function of correctness (Lopez-Martin, Carro, et al. 2020). Their later work extended a related offline RL formulation across several datasets, including CICIDS2017 and UNSW-NB15 (Lopez-Martin, Sanchez-Esguevillas, et al. 2021). Ren et al. used DRL for feature selection and classification on CSE-CIC-IDS2018 (Ren et al. 2022). These works show that RL-style formulations for IDS are precedented.

Other IDS-oriented RL work explores DQN, Double DQN, adversarial-environment DQN, actor-critic, SAC, DDPG, and Rainbow-style designs (Caminero et al. 2019; Han et al. 2020; Hsu et al. 2023; Alavizadeh et al. 2021; Hossain et al. 2025). The details vary widely. Some papers use binary classification, others multi-class attack labels, feature selection, threshold control, or SDN-specific actions. Many still evaluate on static public datasets. That makes them relevant background for the thesis, but it also limits direct comparability. A paper using NSL-KDD with random splits and correctness rewards is not evidence that an offline QRDQN defender will generalize from CICIDS2017 to laboratory traffic.

A second branch studies more genuinely sequential cyber defense. POMDP and attack-graph approaches model defensive actions under uncertainty (Hu et al. 2020). CybORG provides a gym-style environment for autonomous cyber agents (Standen et al. 2021). Broader autonomous cyber defense work discusses agents that select countermeasures in simulated or emulated networks (Palmer et al. 2023; Center for Security and Emerging Technology et al. 2023).

These settings are conceptually different from this thesis because defender actions can alter future state. They represent a stronger form of RL problem than static row classification.

The thesis therefore belongs primarily to the first branch: offline flow-level detection formulated through an RL interface. It can reference autonomous cyber defense as broader context and future direction, but it should not imply that the current system performs multi-agent, adversarial, or live network-control learning. The cleanest wording is that the thesis implements a dataset-as-environment NIDS benchmark with a cost-sensitive binary action interface, not a full cyber-range defender.

3.10 Classification-as-RL and Binary Security Decisions

The `PERMIT/BLOCK` formulation belongs in the classification-as-RL discussion. In the current environment, each flow observation is presented to the agent, the action is binary, and reward is computed from the action and ground-truth label. `PERMIT` corresponds to accepting traffic as benign, while `BLOCK` corresponds to identifying it as attack. A false negative permits an attack flow; a false positive blocks benign traffic.

This framing has a real methodological benefit: it makes cost-sensitive security decisions explicit. IDS errors are not symmetric in practice. Missing an attack and blocking benign traffic have different operational meanings, and the preferred trade-off depends on the defended environment (Lee et al. 2002; Cárdenas et al. 2006). Reward design can encode this asymmetry directly, which is useful for a thesis studying defensive decision rules rather than only label prediction.

The limitation is equally important. If the dataset does not react to the

action, then `PERMIT/BLOCK` is not an active control action. It is an offline decision label over a recorded or extracted flow row. The next row does not change because the agent blocked or permitted the previous row. The formulation is therefore closer to reward-engineered classification than to a firewall, IPS, SDN controller, or autonomous cyber-defense agent. This is not a flaw if stated clearly. It becomes a flaw only if the chapter overclaims live blocking, deployment readiness, or fully sequential control.

The reward function also needs careful interpretation. A positive reward for a true positive and a larger penalty for a false negative can express the idea that missed attacks are costly. A false-positive penalty can express the cost of disrupting benign traffic or overwhelming analysts. A reward of zero or small value for true negatives can keep the focus on attack detection. These are design choices, not universal security economics. Different organizations would choose different costs depending on business continuity, threat model, analyst capacity, and tolerance for missed attacks.

Because the formulation is close to cost-sensitive classification, supervised baselines are mandatory. A strong Random Forest, and ideally other tabular baselines, help determine whether QRDQN adds empirical value beyond the canonical features, preprocessing, and reward design. The same metrics must be computed from both RL and supervised outputs. Training reward alone is not enough, because a reward curve can improve while precision, recall, or false-positive behavior remains operationally poor. The thesis should interpret any RL advantage as benchmark- and protocol-specific unless broader evidence is available.

3.11 Methodological Pitfalls in ML/DL/RL-Based NIDS

The NIDS literature contains repeated warnings against overinterpreting benchmark accuracy. Sommer and Paxson argued that ML-based intrusion detection is difficult to validate in realistic environments because benign traffic is diverse, attacks are rare, labels are hard to obtain, and operational base rates differ sharply from benchmark conditions (Sommer et al. 2010). Arp et al. identify similar risks across machine learning in computer security, including data leakage, sampling bias, inappropriate metrics, and weak experimental design (Arp et al. 2020).

Leakage is broader than accidentally placing the same row in train and test. It includes any target-related information that would not legitimately be available at prediction time (Kaufman et al. 2011). In NIDS, leakage can enter through endpoint identifiers, absolute timestamps, Flow IDs, scenario-specific ports, global preprocessing fitted before splitting, duplicate or near-duplicate flows, and train/test partitions that share capture context. A model that learns that a particular IP, port, day, or Flow ID corresponds to an attack may score well without learning portable malicious behavior.

Temporal and scenario leakage are especially important. Public NIDS datasets are often organized by day, capture file, attack window, or scripted scenario. A random row-wise split can place highly related traffic on both sides of the train/test boundary. Even when rows are not exact duplicates, they may share hosts, attack tools, timing regimes, or extraction artifacts. This is especially important for CICIDS2017 because later studies report errors or artifacts in flow extraction, labeling, and summarized CSV use (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022; Rosay et al. 2022; Dube 2024).

Preprocessing can also leak. Scaling, imputation, feature selection, outlier clipping, or dimensionality reduction must be fitted on the training partition

and then applied to validation or test data. If a scaler is fitted on the full dataset before splitting, distributional information from the test set has entered training. This may look harmless compared with label leakage, but in security datasets with strong day or scenario structure it can still inflate confidence. The repository's use of train-fitted `StandardScaler` objects is therefore part of the methodological design, not only a coding convenience.

Class imbalance and the base-rate problem complicate interpretation. In operational networks, attacks are typically rare relative to benign traffic. A detector can achieve high accuracy while missing many attacks, or can achieve high recall while producing too many false alerts to investigate. Axelsson's base-rate argument remains relevant because even low false-positive rates can dominate alert streams when true attack prevalence is low (Axelsson 2000). This is why accuracy should not be the headline measure for NIDS.

RL introduces additional pitfalls. Deep RL training can be stochastic and sensitive to random seeds, hyperparameters, reward shaping, replay-buffer settings, exploration, and environment details. In a static dataset-as-environment setting, strong results may reflect the same feature and split advantages that benefit supervised models. Therefore, any claim that QRDQN is better than a supervised baseline must be supported by same-protocol comparison and should acknowledge run-to-run variance when repeated seeds are not available. The chapter should treat single-run gains as preliminary unless backed by repeated experiments or strong validation artifacts.

3.12 Evaluation Protocols, Metrics, and Reproducibility

Evaluation should match the strength of the claim. Same-dataset random splits are acceptable for internal learning checks, but they are weak evidence

for deployment. A practical validation ladder for this thesis starts with random stratified splits, then moves to temporal or CSV/day splits, leave-one-scenario or leave-one-exact-CSV-out validation, cross-dataset testing with a harmonized feature schema, and finally independently captured laboratory or production-like traffic. Each step increases distributional separation between training and testing data, and therefore supports stronger generalisation claims.

The current repository reflects this ladder. Random split training verifies that the QRDQN implementation, feature schema, reward function, and training loop can learn the controlled benchmark. Check B with shuffled labels tests whether performance collapses toward a majority-class baseline when labels no longer contain real signal. Check C uses a harder CSV/day split. The leave-one-exact-CSV-out script tests whether holding out one original CICIDS2017 CSV changes behavior. Phase 2 offline inference on laboratory flow CSVs tests the separate question of whether a model trained on the benchmark can process independently extracted flows.

Metrics must also be interpreted operationally. Accuracy is useful but insufficient for imbalanced NIDS. Precision indicates how trustworthy positive alerts are. Recall indicates how much attack traffic is detected. F1 provides a compact balance, but it hides the actual false-positive and false-negative trade-off. False-positive rate maps to benign traffic incorrectly blocked or alerted. False-negative rate maps to malicious traffic missed by the detector (Axelsson 2000; Fawcett 2006; Saito et al. 2015). Confusion matrices, per-class precision, recall, F1, TP, FP, FN, TN, FPR, and FNR should be preferred over a single headline number (Milenkoski et al. 2015).

Cost-sensitive evaluation is relevant because the correct operating point depends on context. NIST guidance and IDS evaluation literature both emphasize trade-offs between false positives and false negatives (Scarfone et al. 2007; Cárdenas et al. 2006). In this thesis, the asymmetric reward design should be treated as a scenario assumption and evaluated empirically, not as a universal cost model for all networks. If the reward penalizes false negatives

heavily, the resulting policy may prefer blocking more flows. That behavior must be judged through precision, recall, false-positive rate, and the specific cost assumptions stated in the experiment.

Reproducibility is part of the evidence, not an administrative detail. Cybersecurity ML results should preserve dataset version, exact dataset variant, feature schema, excluded fields, preprocessing order, split logic, scaler state, random seeds, model configuration, reward configuration, run ID, metrics, and predictions where feasible. Olszewski et al. show that reproducibility remains a major problem in ML security research (Olszewski et al. 2023). The repository's use of saved configs, metrics, model artifacts, validation outputs, and run folders is therefore a methodological strength, but missing artifacts should remain clearly labeled as pending.

This reporting discipline also protects against accidental overclaiming. A result tied to the C03 full random-split QRDQN run is a result for that run, with that split, reward configuration, preprocessing path, and artifact state. It should not silently become a claim about all QRDQN configurations, all CICIDS2017 variants, or all network traffic. The stronger the claim, the more complete the artifact trail must be.

3.13 Data Efficiency and Training-Scale Evaluation

Data efficiency asks how much labeled traffic is needed to reach useful performance, and how the learning curve changes as the training set grows. This question is important for NIDS because labeled attack traffic is expensive, traffic distributions change, and public benchmark abundance does not imply deployment data abundance (Ring et al. 2019; Apruzzese et al. 2022). It is also important for RL because deep RL methods can be sample-hungry, especially

when reward signals are sparse, noisy, delayed, or highly shaped (Sutton et al. 2018; Hessel et al. 2018).

Existing RL-IDS literature often trains on full public datasets and reports final benchmark scores, while controlled training-scale ablations are less common (Yang et al. 2026; Kheddar et al. 2025). Supervised few-shot and class-incremental NIDS work addresses related questions from a different angle (Di Monda et al. 2024), but the thesis question is narrower: under one feature schema and evaluation protocol, how does QRDQN scale with available training rows, and how does that compare with supervised baselines?

The answer matters because model choice and data need are connected. If a Random Forest reaches stable performance with far fewer rows than QRDQN under the same split, that is important evidence even if QRDQN later catches up. If QRDQN is more sensitive to reduced data or to attack-class imbalance, the thesis should report that as a limitation. If QRDQN is competitive at smaller budgets, the evidence should still be tied to the benchmark and validation protocol, not generalized to all NIDS settings.

The thesis positions training-scale experiments as methodological evidence. Budgets such as 100k, 250k, 500k, 1M, and 2M samples can test whether the RL formulation requires substantially more data than a supervised baseline to reach comparable performance. Such experiments should not be framed as proof of few-shot learning or deployment readiness. They are internal evidence about the data requirements of this particular formulation. The useful output is a learning-curve comparison with consistent preprocessing, random seeds where feasible, shared metrics, and explicit artifact paths.

3.14 Research Gap and Positioning of This Thesis

The literature supports a narrow but meaningful research gap. NIDS and flow-based ML are mature areas; public datasets such as CICIDS2017 are useful but fragile; supervised tabular baselines are strong; RL and DRL for IDS already exist; and autonomous cyber defense is a broader, more sequential field than the current implementation (Ring et al. 2019; H. Liu et al. 2019; Yang et al. 2026; Kheddar et al. 2025). The gap is not the absence of RL for intrusion detection.

The contribution of this thesis is a reproducible experimental study of an offline, flow-level, binary security-decision formulation. The system maps CICIDS2017 and later lab-flow inputs into a fixed 76-feature canonical schema, augments observations with a missingness mask to produce 152-dimensional inputs, exposes rows through a Gymnasium-compatible environment, and trains a QRDQN agent to choose between `PERMIT` and `BLOCK`. This makes the cost-sensitive decision interface explicit while preserving a clear comparison obligation against supervised baselines.

The thesis is also positioned methodologically. It treats CICIDS2017 as the main internal benchmark, not as proof of production readiness. It separates random-split results from stricter validation. It uses anti-leakage preprocessing and validation checks to reduce obvious artifact learning. It treats external lab-captured traffic as the preferred next evidential step, while acknowledging that current Phase 2 inference is offline and does not perform active blocking.

The final gap is therefore an integration gap: a leakage-aware, reproducible, cost-sensitive QRDQN benchmark for flow-level NIDS, evaluated against supervised baselines and interpreted through a validation ladder. The thesis can contribute by making the feature contract explicit, saving run artifacts, documenting reward assumptions, comparing against Random Forest under

matched splits, and showing how internal CICIDS2017 performance changes under stricter validation and training-scale constraints.

The defensible final claim is not that QRDQN is universally better than supervised NIDS models, nor that the project implements autonomous cyber defense. The defensible claim is that the thesis evaluates a QRDQN-based, cost-sensitive, flow-level defender under a reproducible benchmark and validation pipeline, and uses supervised baselines, leakage-aware checks, training-scale analysis, and lab-traffic inference to determine how far that formulation can be trusted. If later experiments show weak cross-file or lab-traffic performance, that does not invalidate the thesis. It sharpens its contribution by demonstrating the limits of benchmark-trained RL under more realistic distribution shift.

Chapter 4

Methodology

4.1 Methodological Overview

This chapter describes the methodological design used to build and evaluate the proposed reinforcement learning (RL) based cybersecurity defender. The objective is not to claim a production-ready Intrusion Prevention System (IPS), but to define a controlled, reproducible, offline experimental framework for binary security decisions over network flow records. Each flow is treated as one decision point: the defender observes a numerical representation of the flow and chooses whether to `PERMIT` it as benign traffic or `BLOCK` it as malicious traffic.

The methodology is intentionally conservative. Public intrusion-detection benchmarks can produce inflated results when preprocessing, split construction, leakage controls, and baseline comparisons are weak (Arp et al. 2020; Engelen et al. 2021; Lanvin et al. 2023). Therefore, the pipeline is designed around explicit data contracts, reproducible run artifacts, train-only normalization, and progressively stricter validation. The same canonical representation is used for the CICIDS2017 benchmark and for Phase 2 laboratory-flow inference, so the experimental question is not tied to a single CSV naming convention or a single feature extractor.

At a high level, the methodology contains six linked stages:

1. ingest flow-level CICIDS2017 CSV data and map labels to a binary defender task;
2. remove leakage-prone identifiers and normalize malformed numerical values;
3. map extractor-specific columns into a fixed 76-feature canonical schema;
4. append a 76-feature missingness mask, producing 152-dimensional observations;
5. train and evaluate a Gymnasium-compatible QRDQN defender under controlled split protocols;
6. test robustness through validation checks, supervised baselines, and offline Phase 2 inference.

This design keeps the thesis within an offline experimental boundary. The `PERMIT` and `BLOCK` outputs are model predictions over saved flow records. They are not firewall commands and do not modify any live network path.

4.2 Experimental Phases

The experimental methodology is divided into two phases. The first phase tests learning and internal generalization on a public benchmark. The second phase tests whether the resulting preprocessing and model pipeline can process independently captured traffic under domain shift.

4.2.1 Phase 1: CICIDS2017 Internal Benchmark

Phase 1 uses CICIDS2017 as the main internal benchmark (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017). CICIDS2017 is suitable for

this thesis because it provides labeled benign and attack traffic, raw packet captures, and flow CSVs derived from CICFlowMeter-style extraction (Habibi Lashkari et al. 2017). It also contains multiple attack scenarios across different capture days, which supports stricter split designs than a single random train-test partition.

The phase is not treated as a clean production proxy. CICIDS2017 has known issues involving duplicated or inconsistent flows, label and extraction errors, and split sensitivity (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022). For that reason, Phase 1 is framed as an internal benchmark with explicit controls. The pipeline removes identifier-like fields, fits normalization only on training data, records run configurations, and evaluates the model using a validation ladder. A high score on the random split is interpreted as evidence that the model can learn the benchmark task; it is not interpreted as deployment readiness.

4.2.2 Phase 2: Offline Laboratory-Traffic Inference

Phase 2 evaluates transfer beyond the benchmark distribution. Traffic is captured in a private laboratory environment, converted offline into flow CSVs, mapped into the same canonical schema, normalized with persisted training artifacts, and passed through the trained model. This phase is an external-distribution inference test, not a live deployment.

The methodological purpose of Phase 2 is to expose domain shift. Public NIDS studies often report strong within-dataset results that degrade when the model is evaluated on traffic produced by different networks, capture tools, time periods, or feature definitions (Apruzzese et al. 2022; Layeghy et al. 2023). Phase 2 therefore focuses on reproducible offline inference, diagnostics, and artifact tracking. It does not guarantee stable performance on arbitrary enterprise networks, and it does not implement active real-time blocking.

4.3 Data Source and Input Representation

The thesis uses a flow-level tabular representation. This representation is a deliberate compromise between operational relevance, tractability, and privacy. It captures statistical behavior over packet sequences while avoiding payload inspection.

4.3.1 CICIDS2017 Flow CSVs

The primary data source is the CICIDS2017 flow CSV release. Each CSV row corresponds to a bidirectional network flow summarized by numerical statistics such as duration, packet counts, byte totals, inter-arrival times, packet-length statistics, and TCP flag counts. Flow-based representations are widely used in NIDS research because they compress network behavior into fixed-width numerical rows that can be processed by standard machine-learning methods (Sperotto et al. 2010; Umer et al. 2017).

The local loading pipeline expects the official CICIDS2017 CSV files and supports both full and capped row budgets. Capped budgets allow smoke tests and fast iteration; full budgets support final benchmark runs. The loader reads large CSVs in chunks, standardizes column names by stripping inconsistent whitespace, identifies the label column robustly, and then applies feature cleaning before any split-dependent scaling.

4.3.2 Binary Label Mapping

CICIDS2017 includes several attack labels. The thesis collapses them into a binary defender decision:

- label `0`: `BENIGN`, mapped to the desired action `PERMIT`;
- label `1`: any attack class, mapped to the desired action `BLOCK`.

This binary mapping matches the front-line decision model studied here. It also makes the RL action space simple and auditable. The limitation is that attack-family information is not the primary learning target in the current RL path. Any per-family error analysis must therefore be supported by preserved family labels or separate artifacts; it cannot be inferred from the binary labels alone.

4.3.3 Flow-Level Tabular Representation

The learning substrate is a flow-level table rather than raw packets or payload bytes. This reduces dimensionality and makes large-scale offline training feasible. It also avoids some legal and technical complications of Deep Packet Inspection, especially when payloads are encrypted (Sperotto et al. 2010; Umer et al. 2017). However, flow abstraction loses content, precise packet ordering, and some long-range multi-flow context. The methodology therefore studies a flow-based defender, not a complete network-forensics system.

4.4 Data Cleaning and Preprocessing

Preprocessing is treated as part of the experimental method, not as an implementation detail. In NIDS benchmarks, small preprocessing decisions can change the meaning of the task by leaking labels, suppressing malformed flows, or making test data influence training.

4.4.1 Column Normalization and Numeric Coercion

Raw CSV columns can contain leading spaces, inconsistent capitalization, and mixed data types. The loader first normalizes column names and then coerces feature columns to numerical values. The final feature matrix is represented

as `float32`, while labels are represented as `int64`. This type contract keeps the downstream environment, scaler, and model interface stable.

Rows are not allowed to carry arbitrary strings into the model. Non-numeric feature columns that survive identifier removal are excluded. This prevents accidental introduction of categorical strings whose encoding would be uncontrolled or dataset-specific.

4.4.2 Invalid Values, Missing Values, and Imputation

Network-flow extraction can produce invalid numerical values. Rate features, for example, may become infinite or undefined when computed over a zero or near-zero duration. The pipeline replaces infinities and missing numerical values with zero before canonical mapping. This choice is pragmatic: many flow counters, rates, and aggregate statistics can safely use zero as a neutral placeholder when paired with an explicit missingness signal.

Zero imputation alone would be unsafe because the model could not distinguish a true measured zero from a synthetic placeholder. The canonical schema therefore pairs imputation with a missingness mask. The mask makes feature availability visible to the model and to later diagnostics.

4.4.3 Leakage-Prone Field Exclusion

The preprocessing pipeline enforces an anti-leakage policy grounded in known ML-security evaluation risks (Arp et al. 2020; Kaufman et al. 2011). Source and destination IP addresses, Flow IDs, absolute timestamps, and endpoint identifiers are excluded. Direct port fields are also excluded because they can act as scenario proxies in scripted datasets. For example, an attack day may concentrate malicious traffic on particular services, allowing a model to memorize a capture script rather than learn behavioral flow statistics.

This exclusion is implemented before canonical mapping. The model there-

fore receives only statistical flow features and corresponding mask values. If new datasets or extractors are introduced later, their adapters must preserve the same policy: no identifiers, no absolute time fields, and no ports used directly as label shortcuts.

4.5 Canonical Feature Schema

The canonical schema is the central data contract of the thesis. It separates the model interface from the column names, ordering, and feature availability of a particular extractor.

4.5.1 Canonical Flow Features

The schema defines 76 ordered numerical flow features. They include duration, forward and backward packet counts, byte totals, packet-length statistics, flow and directional inter-arrival-time statistics, packet and byte rates, TCP flag counts, header lengths, window sizes, subflow statistics, and active/idle timing features. These features are chosen because they are flow statistics rather than identifiers, and because they are broadly compatible with CICFlowMeter-style extraction.

For CICIDS2017, an adapter maps original CSV columns into these canonical names. For Phase 2, the robust inference script maps laboratory-flow extractor columns into the same canonical names. The model is therefore trained and evaluated against a fixed feature contract. This design follows the broader methodological motivation of feature standardization in NIDS: cross-domain comparison becomes more auditable when feature definitions are explicit and stable (Sarhan et al. 2021).

Table 4.1 lists the logical groups within the 76-feature schema and the number of features contributed by each group. The grouping is informational;

the model receives all 76 values as a flat vector without structural grouping signals.

Feature Group	Count	Representative Statistics
Flow duration	1	Total duration of the bidirectional flow
Forward packet statistics	12	Packet count, total bytes, mean/std/min/max of packet lengths
Backward packet statistics	12	Same statistics for the reverse direction
Bidirectional aggregates	8	Combined counts, total bytes, bulk-rate averages
Inter-arrival time statistics	10	Flow, forward, and backward IAT mean, std, min, max
TCP flag counts	8	FIN, SYN, RST, PSH, ACK, URG, CWE, ECE counts
Header length aggregates	4	Forward and backward header byte totals
Window size statistics	3	Initial and mean window size per direction
Flow rate features	6	Bytes/s, packets/s per direction and combined
Subflow statistics	6	Subflow packet and byte averages per direction
Active/idle timing	6	Mean, std, max of active and idle period durations

Table 4.1: Logical grouping of the 76 canonical flow features. All groups are merged into a single flat vector before model input.

4.5.2 Missingness Mask

Different extractors may not provide every canonical feature, and even a present source column can contain invalid values. The pipeline therefore produces a 76-dimensional missingness mask. For each canonical feature x_i, the corresponding mask value m_i is:

- $m_i = 1$ when the source feature is present and valid;
- $m_i = 0$ when the feature is absent or had to be imputed.

The mask has two methodological roles. First, it prevents silent imputation: the model can condition its decision on whether a value was observed or filled. Second, it supports Phase 2 diagnostics because a laboratory extractor that lacks many CICIDS2017-style features can be detected through a changed mask pattern rather than hidden inside a dense numerical vector.

4.5.3 Final Observation Vector

The final observation is:

$$o = [x_1, x_2, \ldots, x_{76}, m_1, m_2, \ldots, m_{76}]$$

This produces a 152-dimensional `float32` vector. The first half contains feature values, and the second half contains the mask. The observation size is therefore invariant across CICIDS2017, validation splits, and Phase 2 inference. This invariance is important because the QRDQN policy network expects a fixed input dimension.

4.6 Train-Test Separation and Scaling

The methodology separates fitting and evaluation at every preprocessing stage that can learn from data. This is necessary because leakage can occur not only through columns, but also through statistics such as global means, variances, percentiles, and class distributions.

4.6.1 Random Stratified Split

The random stratified split is the first validation rung. It preserves class proportions across train and test and is useful as a learning-capacity sanity check.

If the model cannot learn under this favorable split, later stricter evaluations are unlikely to succeed.

However, random row-wise splitting is the weakest evidence in this project. Flow datasets can contain correlated records, repeated patterns, and capture-context artifacts. Placing adjacent or highly similar flows on both sides of the split can overstate generalization (Lanvin et al. 2023; Arp et al. 2020). Random-split results are therefore reported as internal benchmark evidence only.

4.6.2 CSV/Day-Based Split

The CSV/day split trains on selected CICIDS2017 files and evaluates on different files. This split tests whether the model transfers across capture days and attack contexts. In the maintained implementation, the default day split trains on Monday, Tuesday, and Wednesday patterns and tests on Thursday and Friday patterns. This is a stricter setting than a random split because entire files are withheld from training.

The day split also aligns with known CICIDS2017 structure: different days contain different benign workloads and attack scenarios. It is therefore better suited for revealing whether the model has learned general flow behavior or day-specific artifacts.

4.6.3 Leave-One-Exact-CSV-Out Split

The strictest internal protocol is leave-one-exact-CSV-out validation. Each fold holds out one official CICIDS2017 CSV as test data and trains on the remaining CSVs. The implemented validation script aggregates per-fold metrics including accuracy, balanced accuracy, attack precision, attack recall, attack F1, benign precision, benign recall, false-positive rate, false-negative rate, block rate, total reward, and time costs.

This protocol is methodologically important because it treats each capture file as a domain-like unit. It is stronger than a substring day split because it uses exact official CSV filenames and avoids accidental overlap. At the time of this chapter, the implementation exists, while final aggregate metrics should only be reported when a committed run artifact is available.

4.6.4 Train-Fitted Standardization

Neural network training is sensitive to feature scale. The pipeline therefore uses `StandardScaler` normalization, fitting the scaler on the training partition only and applying the fitted transform to test data. The same principle is used for persisted inference artifacts: the scaler saved during training is reused for Phase 2, rather than recomputed from laboratory flows.

For robust Phase 2 inference, the training pipeline also persists percentile bounds over the raw canonical features. The Phase 2 inference script can apply raw-feature percentile clipping and post-scaling z-score clipping, then export diagnostics. These controls are not presented as a solution to domain shift; they are safeguards that make extreme out-of-distribution values visible and bounded during offline inference.

4.7 Reinforcement Learning Formulation

The defender is implemented as a Gymnasium-compatible environment (Farama Foundation 2023). This allows standard value-based RL code to interact with the flow dataset through the familiar `reset()` and `step()` interface.

4.7.1 Observation Space

The observation space is a continuous vector space of dimension 152. Each observation represents one scaled canonical flow vector plus its missingness mask. During training, observations are drawn from the training partition. During deterministic evaluation, the environment iterates through the test partition without shuffling, allowing predictions to be compared directly with the true labels.

4.7.2 Action Space

The action space is discrete with two actions:

- `Action 0 (PERMIT)`: classify the flow as benign and allow it in the experimental decision model;
- `Action 1 (BLOCK)`: classify the flow as malicious and block or alert on it in the experimental decision model.

The action value is intentionally aligned with the binary label value: benign is `0`, attack is `1`. This simplifies direct evaluation because a predicted action can be compared with the target label without an additional decoding layer.

4.7.3 Reward Function

The reward function encodes asymmetric security costs (Lee et al. 2002; Cárdenas et al. 2006). A false negative means an attack is permitted. A false positive means benign traffic is blocked. In many defensive settings, missed attacks are more severe than unnecessary blocks, although the exact ratio depends on the operational context.

The current implementation uses the following default reward configura-

tion:

$$\text{TP} = 1.5, \quad \text{FP} = -2.0, \quad \text{FN} = -5.0, \quad \text{TN/omission} = 0.0.$$

Table 4.2 arranges these values as a decision matrix across the four possible outcomes.

Agent Decision	True Label: Attack	True Label: Benign
`BLOCK` (Action 1)	+1.5 (True Positive)	−2.0 (False Positive)
`PERMIT` (Action 0)	−5.0 (False Negative)	0.0 (True Negative)

Table 4.2: Default reward matrix for the binary defender environment. Values are experimental assumptions logged with each run.

The false-negative penalty dominates the reward signal. An agent that issues `PERMIT` on every observation accumulates large negative return on attack-heavy traffic, while an agent that issues `BLOCK` on every observation accumulates false-positive penalties without gaining attack-detection credit. The reward structure is therefore designed to push the agent away from degenerate constant policies and toward discriminative behavior. The true-negative reward of zero is deliberate: explicitly rewarding every correct benign pass would bias the agent toward maximizing pass rate rather than detection quality. These values are experimental assumptions, not measured business costs. They are logged with each run so historical runs can be distinguished from current defaults.

4.7.4 Episode Structure

An episode is a sequence of flow decisions. On reset, the environment initializes an index over the selected partition. When shuffling is enabled, the training order is randomized. At each step, the agent receives one observation, selects `PERMIT` or `BLOCK`, receives a reward computed from the true label and the action, and advances to the next flow. An episode terminates when the dataset

is exhausted or a configured maximum number of steps is reached.

This design gives standard RL algorithms a transition stream while keeping training offline and safe. No action affects the source dataset, the next flow, or the network environment.

4.7.5 Environment State and Transition Protocol

On each `reset()` call the environment selects the partition assigned to the current mode (training or evaluation), optionally shuffles the index if shuffle mode is active, resets the internal step counter, and returns the first observation together with a metadata dictionary. On each `step()` call the environment records the agent's action, looks up the ground-truth label for the current flow index, computes the scalar reward from the reward matrix, advances the index, and returns the next observation, the reward, a termination flag, a truncation flag, and the metadata dictionary.

The termination condition is either dataset exhaustion or a configurable maximum step count. In training mode the shuffled order ensures that mini-batch transitions sample a broad distribution of flow types during a single episode. In evaluation mode the environment is deterministic: the same partition traversed in the same order produces the same sequence of observations, rewards, and labels. This determinism is essential for reproducible evaluation because it means metric differences between two saved models reflect only model behavior rather than sampling noise in the evaluation loop.

The separation between training and evaluation modes is enforced at the environment level rather than left to the caller. Passing the correct mode flag at construction time sets both the index set and the shuffle behavior, which prevents a common mistake where evaluation accidentally inherits shuffled ordering from a training environment.

4.7.6 Limitations of the Dataset-as-Environment Formulation

Because the environment is a static dataset, the agent's actions do not affect future states. This is not a full autonomous cyber-defense problem in which blocking a flow might change later attacker behavior, host state, routing, or alert context. The formulation is closer to a cost-sensitive contextual decision problem than to a rich Markov Decision Process (Sutton et al. 2018; Levine et al. 2020).

This limitation is explicit in the methodology. RL is used here to study value-based cost-sensitive decision learning over flow records and to establish a safe offline pipeline. Claims about multi-step adversary response, online adaptation, or active cyber defense are outside the implemented scope.

4.8 QRDQN Agent

The primary RL algorithm is Quantile Regression Deep Q-Network (QRDQN) (Dabney et al. 2018). QRDQN belongs to the distributional RL family, which models a distribution over returns rather than only an expected return (Bellemare et al. 2017).

4.8.1 Algorithmic Role

Standard DQN approximates action values with a neural network and selects actions according to estimated expected return (Mnih et al. 2015). QRDQN extends this idea by learning quantiles of the return distribution. This is relevant to the thesis because the reward structure is asymmetric: false negatives carry a much larger penalty than correct benign permits carry reward. A distributional view can represent more information about return variability than

a single mean estimate.

The methodology does not assume QRDQN is inherently superior to supervised classifiers for tabular flow data. Instead, it evaluates QRDQN as the main RL representative under the same preprocessing, splits, and metrics used by baselines.

4.8.2 Quantile Regression Loss

QRDQN represents the action-return distribution using N learned quantile estimates $\{\theta_i(s,a)\}_{i=1}^{N}$ at fixed quantile midpoints $\hat{\tau}_i = \frac{2i-1}{2N}$ (Dabney et al. 2018). During action selection, these quantiles are averaged to produce an expected-return estimate, recovering the standard DQN greedy rule under expectation.

For a single transition (s, a, r, s'), let $a^* = \arg\max_a \sum_j \theta_j^-(s', a)$ denote the greedy action under the target network, and let

$$\delta_{ij} = r + \gamma\, \theta_j^-(s', a^*) - \theta_i(s, a)$$

denote the temporal-difference error between source quantile i and bootstrapped target quantile j. QRDQN minimizes the Huber quantile regression loss summed over all N^2 quantile pairs:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{N} \Big| \hat{\tau}_i - \mathbf{1}[\delta_{ij} < 0] \Big| \cdot L_\kappa(\delta_{ij}),$$

where L_κ is the element-wise Huber loss with threshold κ:

$$L_\kappa(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{if } |\delta| \le \kappa, \\ \kappa\Big(|\delta| - \frac{1}{2}\kappa\Big) & \text{otherwise.} \end{cases}$$

Unlike C51, which represents returns over a fixed categorical support, QRDQN places no constraint on the support of the return distribution and adapts to the scale of rewards automatically (Bellemare et al. 2017). In the asymmetric

reward setting of this thesis, the return distribution can develop heavy negative tails when false-negative events are frequent, making a distributional view more informative than a scalar mean estimate for understanding agent behavior near the PERMIT/BLOCK decision boundary.

4.8.3 Exploration Strategy

During training, the agent uses an ε-greedy policy. With probability ε it selects a uniformly random action from $\{\texttt{PERMIT}, \texttt{BLOCK}\}$; with probability $1-\varepsilon$ it selects the action whose average quantile estimate is highest (Mnih et al. 2015). ε decays linearly from an initial value near one toward a small final value over a fixed fraction of the total training timesteps, then remains constant. In the binary action setting the exploration budget is relatively cheap because only two alternatives exist. The exploration fraction and final ε value are stored in the run configuration artifact so that runs with different schedules are not silently merged or compared without accounting for their exploration differences.

4.8.4 Training Configuration

The implementation uses the `sb3-contrib` QRDQN module (Stable-Baselines3 Contributors 2023). The model uses a multilayer perceptron policy over the 152-dimensional observations. Training uses a replay buffer, minibatch updates, a target network, and deterministic prediction during evaluation. The maintained training script supports fast and full presets, random and day split modes, explicit seeds, configurable row caps, and configurable timesteps.

The current full-preset defaults use larger batches and more gradient steps than the fast preset. This separation lets the project support quick smoke runs without confusing them with final benchmark runs. Any reported result must

include the exact run identifier and configuration.

4.8.5 Model and Preprocessing Persistence

Training produces a run directory and a model artifact. The run directory stores configuration, metrics, the fitted scaler, and training percentile bounds. Persisting preprocessing artifacts is essential because Phase 2 inference must use the same normalization state as training. Recomputing a scaler on laboratory traffic would leak inference-distribution information into the model interface and would make the run irreproducible.

4.9 Supervised Baseline

An RL method for flow-level NIDS must be compared against strong supervised baselines. Tabular flow data is a setting where tree-based models, especially Random Forests and gradient-boosted ensembles, are often competitive (H. Liu et al. 2019; Apruzzese et al. 2022).

4.9.1 Random Forest Baseline

The primary supervised baseline is a Random Forest classifier. The baseline is methodologically useful because it is strong on tabular data, handles nonlinear feature interactions, and requires less training complexity than deep RL. It also provides a sanity check against over-valuing an RL formulation in a task that may behave like cost-sensitive classification.

4.9.2 Same-Protocol Comparison

The baseline must consume the same canonical observation representation, use the same split protocols, and report the same metrics as QRDQN. This same-protocol rule prevents confounded comparisons. If the baseline uses different preprocessing, different leakage controls, or easier splits, the comparison does not isolate the algorithmic contribution of QRDQN.

The methodology therefore treats baseline comparison as a protocol requirement rather than an optional appendix. It also avoids claiming QRDQN superiority until same-split baseline artifacts exist.

4.9.3 Class Imbalance and Cost-Sensitive Weighting

CICIDS2017 is class-imbalanced: benign flows typically outnumber attack flows substantially, and within attack flows individual families vary in prevalence. A Random Forest trained with default uniform class weights will therefore tend to bias its predictions toward the majority benign class. To make the comparison meaningful under the asymmetric cost assumptions of the thesis, the Random Forest baseline uses class-balanced weighting, where each class receives a weight inversely proportional to its frequency in the training partition. This ensures that the supervised model is not penalized in the same direction as the RL reward function for different reasons.

The class-weight setting is recorded in the baseline configuration artifact alongside the split mode, feature schema, and evaluation seed, so the weighting assumption is traceable. If the thesis finds that the Random Forest under balanced weighting reaches similar or superior false-negative performance to QRDQN under the asymmetric reward, that comparison is informative precisely because both models have been given access to comparable cost-sensitivity mechanisms. Comparing a cost-adjusted QRDQN against an uncorrected Random Forest would not provide this evidence.

4.10 Training-Scale and Data-Efficiency Protocol

The methodology includes a training-scale component motivated by a practical question: how much labeled flow data does each approach require before reaching stable performance? This matters because labeled attack traffic is expensive to obtain, benchmark abundance does not reflect deployment availability, and deep RL methods can be substantially more sample-hungry than tree-based supervised learners (Sutton et al. 2018; Ring et al. 2019).

4.10.1 Learning Curve Protocol

The learning-curve protocol evaluates both the QRDQN agent and the Random Forest baseline at progressively larger training budgets, all drawn from the same stratified split. The evaluation set is held constant across budgets at the full test partition so that metric changes reflect only training-data size and not evaluation conditions. Metrics are recorded at each budget level using the same evaluation code as the main benchmark runs.

Each budget uses a freshly fitted scaler computed from only the rows included in that training subset. Larger budgets therefore benefit from a more representative normalization without contaminating the fixed test evaluation. This means the preprocessing artifacts produced at each budget level are distinct and independently reproducible.

4.10.2 Budget Levels and Comparison

Candidate budget levels include 100 k, 250 k, 500 k, 1 M, and 2 M flow records. Not every level is achievable under every split mode; when a split does not produce training sets of the target size, the nearest available size is recorded.

The final budget is the complete training partition under the selected split.

Comparing QRDQN and Random Forest at each budget level reveals whether the RL formulation has a different sample requirement or convergence trajectory. If the Random Forest reaches a stable performance plateau with substantially fewer rows, this constitutes internal evidence that the benchmark task does not require the additional sample complexity of deep RL on the current feature schema. Conversely, if the cost-sensitive RL behavior improves disproportionately as the budget grows, the training-scale results can identify at what scale the distributional reward structure starts to contribute measurable advantage. All training-scale results are stored under the same run artifact scheme as main experiments, labeled by budget level and seed, so they remain independently recoverable and comparable across future experiments.

4.11 Evaluation Outputs and Reproducibility

Reproducibility is a core design constraint, especially because ML-security research often suffers from missing code, missing data processing details, and incomplete evaluation records (Olszewski et al. 2023).

4.11.1 Metrics

The primary binary metrics are accuracy, precision, recall, and F1 for both benign and attack classes. The analysis also tracks confusion-matrix counts: true positives, false positives, false negatives, and true negatives. These counts are required because two models with similar aggregate accuracy can have very different security implications.

False-positive rate and false-negative rate are especially important. The false-positive rate measures the share of benign traffic incorrectly blocked, which relates to availability and analyst workload. The false-negative rate

measures the share of attacks incorrectly permitted, which relates directly to missed intrusion risk. Precision-recall analysis is particularly relevant under class imbalance, where accuracy alone can be misleading (Fawcett 2006; Saito et al. 2015).

4.11.2 Validation Ladder

The evaluation ladder is ordered from weakest to strongest evidence:

1. random stratified split, used as a learning-capacity check;
2. direct prediction check, which verifies model outputs against test labels without depending on environment metadata;
3. shuffled-label check, which helps detect leakage by confirming that performance collapses when training labels are randomized;
4. CSV/day split, which withholds entire capture groups;
5. leave-one-exact-CSV-out validation, which withholds each official CSV in turn;
6. Phase 2 laboratory-flow inference, which tests the trained pipeline on independently captured traffic.

Each rung targets a different failure mode. The ladder does not make any single result definitive, but it makes the evaluation more informative than a single random train-test score.

The random stratified split tests whether the model and implementation can solve the benchmark at all. A model that fails here has a training or architecture problem, not a generalization problem. The shuffled-label check is a leakage detector: a model that achieves well above majority-class accuracy on a shuffled-label run has learned from label-correlated artifacts in the features rather than from true flow statistics (Arp et al. 2020). The CSV/day

split tests temporal and scenario generalization by withholding entire capture groups; performance degradation relative to the random split reveals whether the model depends on day-specific context. The leave-one-exact-CSV-out protocol pushes further: each official CICIDS2017 file is held out in turn, and the aggregate metric distribution shows whether any single file dominates the benchmark or whether performance is stable across capture scenarios. Phase 2 inference is the final and strictest rung, testing whether the entire pipeline can operate on traffic produced outside the benchmark environment.

4.11.3 Run Artifacts

Every meaningful training or evaluation run should persist a unique `RUN_ID`, a configuration file, metrics, and any preprocessing state needed for exact reuse. Training runs persist model weights and scaler artifacts. Validation runs persist validation results. Phase 2 inference runs persist predictions, metrics, configuration, and optional diagnostics.

This artifact discipline prevents ambiguous claims. A result is only treated as measured when it can be tied to a specific run folder and configuration. Historical runs are not silently merged with current defaults; reward values, split logic, preprocessing, row caps, and seeds must be read from the artifact.

4.12 Phase 2 Offline Inference Pipeline

The maintained Phase 2 entry point is the robust offline inference script. It accepts an extracted flow CSV, a trained model, a persisted scaler, optional percentile bounds, and optional z-score clipping. It then maps input columns to the canonical schema, applies the same feature/mask representation, scales with the saved scaler, computes diagnostics, predicts actions in batches, and writes outputs under a new Phase 2 run directory.

4.12.1 Canonical Mapping for Laboratory Flows

Laboratory-flow extractors may use different column names from CICIDS2017. The Phase 2 script therefore defines an explicit mapping from extractor names to canonical names. Metadata columns such as source IP, destination IP, ports, protocol, and timestamp may be preserved in output files for inspection, but they are not part of the model feature vector. This keeps interpretability and model input separate.

4.12.2 Distribution-Shift Diagnostics

Phase 2 inference computes diagnostics over scaled canonical features. Large absolute z-scores indicate that laboratory flow values are far from the CICIDS2017 training distribution. The script can also export the features with the largest deviations. These diagnostics are not labels and do not prove correctness, but they help explain block-rate changes and identify mismatches between benchmark and laboratory traffic.

Beyond per-feature z-scores, the diagnostic output records the proportion of canonical features that were imputed from the missingness mask, the mean and variance of scaled feature values relative to training statistics, and any features that consistently fall outside the percentile bounds observed during CICIDS2017 training. Together these indicators provide a structured summary of distributional distance between the laboratory traffic and the benchmark, enabling targeted investigation rather than requiring manual inspection of raw prediction logs.

4.12.3 Truth Metrics When Available

If a Phase 2 flow CSV includes truth columns such as `truth_y` or `truth_label`, the script can compute the same binary metrics used in internal evaluation.

If ground truth is absent, the script reports prediction statistics such as block rate, allow rate, and z-score diagnostics. This distinction is important: an unlabeled benign-only or mixed-lab run cannot support the same claims as a labeled external validation set.

4.13 Implementation Framework

The experimental pipeline is implemented in Python and relies on a set of well-established open-source libraries. The core deep RL component uses the `sb3-contrib` extension of Stable Baselines3, which provides a tested QRDQN implementation with configurable network architecture, replay buffer, exploration schedule, and target-network updates (Stable-Baselines3 Contributors 2023). The environment wrapper follows the Gymnasium API (Farama Foundation 2023), making the flow dataset compatible with any Gymnasium-compliant training loop without changes to the algorithm implementation.

4.13.1 Data Processing and Supervised Learning Stack

Data loading and preprocessing rely on `pandas` for CSV ingestion, column standardization, and chunked reading of large files. Feature matrices are converted to `numpy` arrays and cast to `float32` before entering the Gymnasium environment or the supervised baseline. Standardization uses `scikit-learn`'s `StandardScaler`, chosen for its compatibility with the `sklearn` serialization protocol, which allows fitted scalers to be persisted and reloaded without reimplementation. The Random Forest baseline also uses `scikit-learn`, consuming the same input arrays that enter the RL environment. This shared data path ensures that any performance difference between the two models reflects algorithmic behavior rather than a discrepancy in data handling.

4.13.2 Artifact Management

Each training run produces a unique `RUN_ID` composed of a timestamp and a short random suffix. The run directory collects the serialized QRDQN policy, the fitted scaler, percentile bounds, a configuration JSON, and a metrics JSON. Validation and Phase 2 inference runs produce separate subdirectories with their own configuration and metric records. This flat directory scheme avoids database dependencies while making runs independently recoverable and comparable through file-system inspection.

4.13.3 Reproducibility Controls

Seeds are fixed for the `numpy`, `random`, and `torch` random-number generators at the start of each run. Gymnasium environment resets also receive an explicit seed. Where multiple seeds are used for variance estimation, each seed generates a separate run directory so that per-seed results are preserved individually rather than averaged silently. These controls reduce within-run variance enough to distinguish meaningful performance differences from noise, though they cannot eliminate all nondeterminism arising from GPU kernel ordering.

4.14 Methodological Limitations

The methodology makes deliberate trade-offs that define the correct interpretation of the thesis results. This section organizes those trade-offs into four categories.

4.14.1 Static Environment and Sequential Decision-Making

The most fundamental limitation is that the RL environment is a static dataset. The agent's actions do not alter future flow observations, the network topology, the attacker's behavior, or the system state. The formulation is therefore closer to a cost-sensitive contextual decision problem than to a full Markov Decision Process (Sutton et al. 2018; Levine et al. 2020). Sequential dependencies, multi-stage attack chains, attacker adaptation, and active defender countermeasures are all outside the implemented scope. Claims about sequential autonomy, adaptive cyber defense, or online reinforcement learning are not supported by the current pipeline.

4.14.2 Binary Label Mapping and Attack-Family Detail

The binary label mapping collapses all attack types into a single `BLOCK` class. Attack-family information is discarded in the main RL learning path. This design is justified by the binary decision model studied here, but it means the agent cannot specialize its detection quality for individual attack families. Per-family error analysis requires preserved attack labels and separate evaluation code and cannot be inferred from the binary accuracy metrics reported by the main pipeline.

4.14.3 Benchmark Fragility and Generalization

CICIDS2017 remains a controlled public benchmark with documented issues: extraction artifacts, label inconsistencies, duplicate flows, and split sensitivity (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022). Even with anti-leakage preprocessing and stricter split protocols, performance on CICIDS2017 is not equivalent to performance on a production network. Phase 2 laboratory

inference tests domain shift but is offline inference over a private capture rather than validation on an independently labeled production stream. Stronger generalization evidence would require external labeled datasets, multiple capture environments, and eventually a safety analysis for any active deployment.

4.14.4 Single Algorithm and Reproducibility Scope

The primary RL algorithm is a single QRDQN configuration. Single-run results cannot quantify run-to-run variance unless multiple seeds are evaluated and their results aggregated (Sutton et al. 2018). The supervised baseline is limited to Random Forest in the core comparison; other tabular learners are discussed in the literature review but are not part of the evaluated protocol. These scope restrictions are practical: completing the core pipeline and validation ladder is more methodologically informative than partially implementing many comparisons. Stronger future claims would require completed leave-one-exact-CSV-out artifacts, same-protocol supervised baseline metrics, multiple seeds, richer labeled laboratory traffic, and eventually a separate safety analysis for any active deployment.

Chapter 5

Bibliography

[1] Amrin Maria Khan Adawadkar and Nilima Kulkarni. "Cyber-Security and Reinforcement Learning: A Brief Survey". In: *Engineering Applications of Artificial Intelligence* 114 (2022), p. 105116. DOI: 10.1016/j.engappai.2022.105116 (cit. on p. 33).

[2] Zeeshan Ahmad, Adnan Shahid Khan, Cheah Wai Shiang, Johari Abdullah, and Farhan Ahmad. "Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches". In: *Transactions on Emerging Telecommunications Technologies* 32.1 (2021), e4150. DOI: 10.1002/ett.4150 (cit. on p. 28).

[3] Hooman Alavizadeh, Julian Jang-Jaccard, and Hootan Alavizadeh. "Deep Q-Learning Based Reinforcement Learning Approach for Network Intrusion Detection". In: (2021). arXiv: 2111.13978 [cs.CR] (cit. on p. 34).

[4] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood, and Adnan Anwar. "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems". In: *IEEE Access* 8 (2020), pp. 165130–165150. DOI: 10.1109/ACCESS.2020.3022862 (cit. on p. 25).

[5] Giovanni Apruzzese, Luca Pajola, and Mauro Conti. "The Cross-Evaluation of Machine Learning-Based Network Intrusion Detection Systems". In: *IEEE Transactions on Network and Service Management* 19.4 (2022), pp. 5152–5169. DOI: 10.1109/TNSM.2022.3157344 (cit. on pp. 3, 12, 14, 24, 28, 40, 46, 61).

[6] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. "Dos and Don'ts of Machine Learning in Computer Security". In: (2020). arXiv: 2010.09470 [cs.CR] (cit. on pp. 3, 13, 23, 29, 37, 44, 49, 53, 65).

[7] Momand Asadullah, Jan Sana Ullah, and Naeem Ramzan. "A Systematic and Comprehensive Survey of Recent Advances in Intrusion Detection Systems Using Machine Learning: Deep Learning, Datasets, and Attack Taxonomy". In: *Expert Systems* (2023). DOI: 10.1155/2023/6048087 (cit. on p. 29).

[8] Stefan Axelsson. "The Base-Rate Fallacy and the Difficulty of Intrusion Detection". In: *ACM Transactions on Information and System Security* 3.3 (2000), pp. 186–205. DOI: 10.1145/357802.357804 (cit. on pp. 2, 38, 39).

[9] Marc G. Bellemare, Will Dabney, and Rémi Munos. "A Distributional Perspective on Reinforcement Learning". In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. 2017, pp. 449–458. URL: https://proceedings.mlr.press/v70/bellemare17a.html (cit. on pp. 32, 33, 58, 59).

[10] Guillermo Caminero, Manuel Lopez-Martin, and Belen Carro. "Adversarial Environment Reinforcement Learning Algorithm for Intrusion Detection". In: *2019 International Joint Conference on Neural Networks (IJCNN)*. 2019, pp. 1–8. DOI: 10.1109/IJCNN.2019.8852380 (cit. on p. 34).

[11] Canadian Institute for Cybersecurity. *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. Dataset documentation page. 2017. URL: https://www.unb.ca/cic/datasets/ids-2017.html (cit. on pp. 5, 22, 26, 45).

[12] Alvaro A. Cárdenas, John S. Baras, and Karl Seamon. "A Framework for the Evaluation of Intrusion Detection Systems". In: *2006 IEEE Symposium on Security and Privacy (S&P)*. 2006, pp. 63–77. DOI: 10.1109/SP.2006.2 (cit. on pp. 5, 11, 35, 39, 55).

[13] Center for Security and Emerging Technology and The Alan Turing Institute. *Autonomous Cyber Defense*. https://cset.georgetown.edu/publication/autonomous-cyber-defense/. 2023 (cit. on p. 34).

[14] Communications Security Establishment and Canadian Institute for Cybersecurity. *A Realistic Cyber Defense Dataset (CSE-CIC-IDS2018)*. 2018. URL: https://registry.opendata.aws/cse-cic-ids2018/ (cit. on pp. 24, 25).

[15] Will Dabney, Mark Rowland, Marc G. Bellemare, and Rémi Munos. "Distributional Reinforcement Learning with Quantile Regression". In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, pp. 2892–2901. URL: https://ojs.aaai.org/index.php/AAAI/article/view/11791 (cit. on pp. 2, 4, 11, 32, 33, 58, 59).

[16] Davide Di Monda et al. "Few-Shot Class-Incremental Learning for Network Intrusion Detection Systems". In: (2024). Exact venue and DOI require verification; see arXiv preprint search for Di Monda 2024 (cit. on p. 41).

[17] Rohit Dube. "Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research". In: *Journal of Computer Virology and Hacking Techniques* 20.1 (2024), pp. 203–211. DOI: 10.1007/s11416-023-00509-7 (cit. on pp. 27, 37).

[18] Gints Engelen, Vera Rimmer, and Wouter Joosen. “Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study”. In: *2021 IEEE Security and Privacy Workshops (SPW)*. 2021, pp. 7–12. DOI: 10.1109/SPW53761.2021.00009 (cit. on pp. 3, 5, 25, 26, 37, 44, 46, 70).

[19] Farama Foundation. *Gymnasium Documentation*. Software documentation; access date 2026. 2023. URL: https://gymnasium.farama.org/ (cit. on pp. 5, 10, 33, 54, 68).

[20] Tom Fawcett. “An Introduction to ROC Analysis”. In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874. DOI: 10.1016/j.patrec.2005.10.010 (cit. on pp. 39, 65).

[21] Scott Fujimoto, David Meger, and Doina Precup. “Off-Policy Deep Reinforcement Learning without Exploration”. In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research. 2019, pp. 2052–2062. URL: https://proceedings.mlr.press/v97/fujimoto19a.html (cit. on p. 32).

[22] Afrah Gueriani, Hamza Kheddar, and Ahmed Cherif Mazari. “Deep Reinforcement Learning for Intrusion Detection in IoT: A Survey”. In: (2024). Preprint; published version in IC2EM proceedings 2023. arXiv: 2405.20038 [cs.CR] (cit. on pp. 19, 33).

[23] Arash Habibi Lashkari, Gerard Draper Gil, Mohammad Saiful Islam Mamun, and Ali A. Ghorbani. “Characterization of Tor Traffic Using Time Based Features”. In: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*. 2017, pp. 253–262. DOI: 10.5220/0006105602530262 (cit. on pp. 22, 26, 46).

[24] Dongkun Han, Zhiping Wang, Yihua Zhong, Wenhan Chen, Jiaze Yang, Shanqing Lu, Xinyan Shi, and Xia Yin. “Evaluating Network Intrusion Detection Systems for Different Vulnerability Conditions Using an Adversarial Environment DQN”. In: *IEEE Access* 9 (2020). IEEE Xplore

document ID 9289884; full author list requires verification, pp. 4345–4359. DOI: 10.1109/ACCESS.2020.3047430 (cit. on p. 34).

[25] Hado van Hasselt, Arthur Guez, and David Silver. "Deep Reinforcement Learning with Double Q-Learning". In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*. 2016, pp. 2094–2100. arXiv: 1509.06461 (cit. on p. 32).

[26] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar, and David Silver. "Rainbow: Combining Improvements in Deep Reinforcement Learning". In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, pp. 3215–3222. URL: https://ojs.aaai.org/index.php/AAAI/article/view/11796 (cit. on pp. 32, 41).

[27] Md. Alamgir Hossain et al. "Deep Q-Learning Intrusion Detection System (DQ-IDS): A Novel Reinforcement Learning Approach for Adaptive and Self-Learning Cybersecurity". In: *ICT Express* (2025). Full author list requires verification; accessed via ScienceDirect. DOI: 10.1016/j.icte.2025.02.006 (cit. on p. 34).

[28] Chia-Wei Hsu, Chia-Hsuan Lin, et al. "A Soft Actor-Critic Reinforcement Learning Algorithm for Network Intrusion Detection". In: *Computers & Security* 136 (2023). Full author list requires verification against published paper, p. 103580. DOI: 10.1016/j.cose.2023.103580 (cit. on p. 34).

[29] Zhisheng Hu, Minghui Zhu, and Peng Liu. "Adaptive Cyber Defense Against Multi-Stage Attacks Using Learning-Based POMDP". In: *ACM Transactions on Privacy and Security* 24.1 (2020), pp. 1–31. DOI: 10.1145/3418897 (cit. on p. 34).

[30] Shachar Kaufman, Saharon Rosset, and Claudia Perlich. “Leakage in Data Mining: Formulation, Detection, and Avoidance”. In: *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2011, pp. 556–563. DOI: 10.1145/2020408.2020496 (cit. on pp. 3, 13, 23, 37, 49).

[31] KDD Cup. *KDD Cup 1999 Data*. Dataset page. 1999. URL: http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html (cit. on pp. 24, 25).

[32] Hamza Kheddar, Diana W. Dawoud, Ali Ismail Awad, Yassine Himeur, and Muhammad Khurram Khan. “Reinforcement-Learning-Based Intrusion Detection in Communication Networks: A Review”. In: *IEEE Communications Surveys & Tutorials* 27.4 (2025), pp. 2420–2469. DOI: 10.1109/COMST.2024.3484491 (cit. on pp. 19, 33, 41, 42).

[33] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. “Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset”. In: *Future Generation Computer Systems* 100 (2019), pp. 779–796. DOI: 10.1016/j.future.2019.05.041 (cit. on p. 25).

[34] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Éric Totel. “Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes”. In: *Risks and Security of Internet and Systems*. Vol. 13857. Lecture Notes in Computer Science. Springer, 2023, pp. 19–34. DOI: 10.1007/978-3-031-31108-6_2 (cit. on pp. 3, 6, 25, 27, 37, 44, 46, 53, 70).

[35] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Éric Totel. “Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research”. In: *Risks and Security of Internet and Systems*. Vol. 13561. Lecture Notes in Computer Science.

Earlier presentation of dataset-critique work; see also Lanvin2023Errors. Springer, 2022 (cit. on p. 27).

[36] Siamak Layeghy and Marius Portmann. “Explainable Cross-Domain Evaluation of ML-Based Network Intrusion Detection Systems”. In: *Computers and Electrical Engineering* 108 (2023), p. 108692. DOI: 10.1016/j.compeleceng.2023.108692 (cit. on pp. 14, 24, 46).

[37] Wenke Lee, Wei Fan, Matthew Miller, Salvatore J. Stolfo, and Erez Zadok. “Toward Cost-Sensitive Modeling for Intrusion Detection and Response”. In: *Journal of Computer Security* 10.1–2 (2002), pp. 5–22. DOI: 10.3233/JCS-2002-101-202 (cit. on pp. 2, 11, 35, 55).

[38] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. “Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems”. In: *arXiv preprint arXiv:2005.01643* (2020). Preprint. arXiv: 2005.01643 [cs.LG] (cit. on pp. 4, 32, 58, 70).

[39] Long-Ji Lin. “Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching”. In: *Machine Learning* 8.3–4 (1992), pp. 293–321. DOI: 10.1007/BF00992699 (cit. on p. 31).

[40] Hongyu Liu and Bo Lang. “Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey”. In: *Applied Sciences* 9.20 (2019), p. 4396. DOI: 10.3390/app9204396 (cit. on pp. 3, 12, 28, 29, 42, 61).

[41] Lisa Liu, Gints Engelen, Timothy M. Lynar, Daryl Essam, and Wouter Joosen. “Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018”. In: *2022 IEEE Conference on Communications and Network Security (CNS)*. 2022, pp. 254–262. DOI: 10.1109/CNS56114.2022.9947235 (cit. on pp. 25, 27, 37, 46, 70).

[42] Manuel Lopez-Martin, Belén Carro, and Antonio Sanchez-Esguevillas. “Application of Deep Reinforcement Learning to Intrusion Detection for

Supervised Problems". In: *Expert Systems with Applications* 141 (2020), p. 112963. DOI: 10.1016/j.eswa.2019.112963 (cit. on p. 34).

[43] Manuel Lopez-Martin, Antonio Sanchez-Esguevillas, Juan I. Arribas, and Belén Carro. "Network Intrusion Detection Based on Extended RBF Neural Network With Offline Reinforcement Learning". In: *IEEE Access* 9 (2021), pp. 153153–153170. DOI: 10.1109/ACCESS.2021.3127689 (cit. on p. 34).

[44] Aleksandar Milenkoski, Marco Vieira, Samuel Kounev, Alberto Avritzer, and Bryan D. Payne. "Evaluating Computer Intrusion Detection Systems: A Survey of Common Practices". In: *ACM Computing Surveys* 48.1 (2015), 12:1–12:41. DOI: 10.1145/2808691 (cit. on p. 39).

[45] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. "Human-Level Control through Deep Reinforcement Learning". In: *Nature* 518.7540 (2015), pp. 529–533. DOI: 10.1038/nature14236 (cit. on pp. 32, 58, 60).

[46] Nour Moustafa, Mohiuddin Ahmed, and Sherif Ahmed. "Data Analytics-Enabled Intrusion Detection: Evaluations of ToN_IoT Linux Datasets". In: *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 2020, pp. 727–735. DOI: 10.1109/TrustCom50675.2020.00080 (cit. on p. 25).

[47] Nour Moustafa and Jill Slay. "The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set". In: *Information Security Journal: A Global Perspective* 25.1-3 (2016), pp. 18–31. DOI: 10.1080/19393555.2015.1125974 (cit. on p. 24).

[48] Nour Moustafa and Jill Slay. “UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set)”. In: *2015 Military Communications and Information Systems Conference (MilCIS)*. 2015, pp. 1–6. DOI: 10.1109/MilCIS.2015.7348942 (cit. on pp. 24, 25).

[49] Loc Gia Nguyen and Kohei Watabe. “Flow-Based Network Intrusion Detection Based on BERT Masked Language Model”. In: *CoNEXT 2022 Student Workshop*. 2022. DOI: 10.1145/3565477.3569152. arXiv: 2306.04920 (cit. on p. 29).

[50] Daniel Olszewski, Allison Lu, Carson Stillman, Kevin Warren, Cole Kitroser, Alejandro Pascual, Divyajyoti Ukirde, Kevin Butler, and Patrick Traynor. ““Get in Researchers; We’re Measuring Reproducibility”: A Reproducibility Study of Machine Learning Papers in Tier 1 Security Conferences”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2023, pp. 3433–3459. DOI: 10.1145/3576915.3623130 (cit. on pp. 40, 64).

[51] Gregory Palmer, Chris Parry, Daniel J. B. Harrold, and Chris Willis. “Deep Reinforcement Learning for Autonomous Cyber Defence: A Survey”. In: (2023). arXiv: 2310.07745 [cs.CR] (cit. on p. 34).

[52] Kezhou Ren, Jinshu Zhu, Tao Yuan, Sheng Wen, and Frank Jiang. “ID-RDRL: A Deep Reinforcement Learning-Based Feature Selection Intrusion Detection Model”. In: *Scientific Reports* 12 (2022), p. 15370. DOI: 10.1038/s41598-022-19366-3 (cit. on p. 34).

[53] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. “A Survey of Network-Based Intrusion Detection Data Sets”. In: *Computers & Security* 86 (2019), pp. 147–167. DOI: 10.1016/j.cose.2019.06.005. arXiv: 1903.02460 (cit. on pp. 3, 13, 20, 24, 25, 29, 40, 42, 63).

[54] Arnaud Rosay, Eloïse Cheval, Florent Carlier, and Pascal Leroux. "Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017". In: *Proceedings of the 8th International Conference on Information Systems Security and Privacy (ICISSP)*. 2022, pp. 25–36. DOI: 10.5220/0010774000003120 (cit. on pp. 27, 37).

[55] Takaya Saito and Marc Rehmsmeier. "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets". In: *PLOS ONE* 10.3 (2015), e0118432. DOI: 10.1371/journal.pone.0118432 (cit. on pp. 39, 65).

[56] Mohanad Sarhan, Siamak Layeghy, and Marius Portmann. "Evaluating Standard Feature Sets Towards Increased Generalisability and Explainability of ML-Based Network Intrusion Detection". In: (2021). Proposes standardised NetFlow-aligned feature mapping across CIC and UNSW-NB15 datasets. arXiv: 2104.07183 [cs.CR] (cit. on pp. 23, 30, 50).

[57] Habeeb Mohammed Sayeeduddin and T. Ranga Babu. "Network Intrusion Detection System: A Survey on Artificial Intelligence-Based Techniques". In: *Expert Systems* (2022). DOI: 10.1111/exsy.13066 (cit. on p. 29).

[58] Karen A. Scarfone and Peter M. Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. Tech. rep. Special Publication 800-94. National Institute of Standards and Technology, 2007. URL: https://csrc.nist.gov/pubs/sp/800/94/final (cit. on pp. 1, 19, 20, 39).

[59] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. "Prioritized Experience Replay". In: *International Conference on Learning Representations*. 2016. arXiv: 1511.05952 (cit. on p. 32).

[60] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. "A Detailed Analysis of the CICIDS2017 Data Set". In: *Information Systems Security and Privacy*. Vol. 977. Communications in Computer and In-

formation Science. Springer, 2019, pp. 172–188. DOI: 10.1007/978-3-030-25109-3_9 (cit. on p. 26).

[61] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization". In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 2018, pp. 108–116. DOI: 10.5220/0006639801080116. URL: https://www.unb.ca/cic/datasets/ids-2017.html (cit. on pp. 5, 22, 24–26, 45).

[62] Robin Sommer and Vern Paxson. "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection". In: *2010 IEEE Symposium on Security and Privacy (S&P)*. 2010, pp. 305–316. DOI: 10.1109/SP.2010.25 (cit. on pp. 20, 37).

[63] Anna Sperotto, Gregor Schaffrath, Ramin Sadre, Cristian Morariu, Aiko Pras, and Burkhard Stiller. "An Overview of IP Flow-Based Intrusion Detection". In: *IEEE Communications Surveys & Tutorials* 12.3 (2010), pp. 343–356. DOI: 10.1109/SURV.2010.032210.00054 (cit. on pp. 2, 9, 22, 47, 48).

[64] Stable-Baselines3 Contributors. *QR-DQN: Stable Baselines3 Contrib Documentation*. Software documentation; access date 2026. 2023. URL: https://sb3-contrib.readthedocs.io/en/master/modules/qrdqn.html (cit. on pp. 5, 11, 33, 60, 68).

[65] Maxwell Standen, Martin Lucas, David Bowman, Toby J. Richer, Junae Kim, and Damian Marriott. "CybORG: A Gym for the Development of Autonomous Cyber Agents". In: *arXiv preprint arXiv:2108.09118* (2021). Preprint; also cited as an IJCAI-21 Adaptive Cyber Defense workshop paper. arXiv: 2108.09118 [cs.CR] (cit. on p. 34).

[66] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. URL: http://incompleteideas.net/book/the-book-2nd.html (cit. on pp. 2, 30, 41, 58, 63, 70, 71).

[67] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. "A Detailed Analysis of the KDD CUP 99 Data Set". In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. 2009, pp. 1–6. DOI: 10.1109/CISDA.2009.5356528 (cit. on pp. 24, 25).

[68] Ashish Thakkar and Rachna Lohiya. "A Review of the Advancement in Intrusion Detection Datasets". In: *Procedia Computer Science* 167 (2020), pp. 636–645. DOI: 10.1016/j.procs.2020.03.270 (cit. on p. 24).

[69] Muhammad Fahad Umer, Muhammad Sher, and Yaxin Bi. "Flow-Based Intrusion Detection: Techniques and Challenges". In: *Computers & Security* 70 (2017), pp. 238–254. DOI: 10.1016/j.cose.2017.05.009 (cit. on pp. 2, 9, 22, 47, 48).

[70] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. "Dueling Network Architectures for Deep Reinforcement Learning". In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. Vol. 48. 2016, pp. 1995–2003. URL: https://proceedings.mlr.press/v48/wangf16.html (cit. on p. 32).

[71] Christopher J. C. H. Watkins and Peter Dayan. "Q-Learning". In: *Machine Learning* 8.3–4 (1992), pp. 279–292. DOI: 10.1007/BF00992698 (cit. on p. 31).

[72] Jianping Xiao, Chun Long, Jing Zhao, Jinxia Wei, Anlei Hu, and Guanyao Du. "A Survey on Network Intrusion Detection Based on Deep Learning". In: *Frontiers of Data and Computing* 3.3 (2021), pp. 59–74. DOI: 10.11871/jfdc.issn.2096-742X.2021.03.006 (cit. on p. 29).

[73] Wanrong Yang, Alberto Acuto, Yihang Zhou, and Dominik Wojtczak. "A Survey for Deep Reinforcement Learning Based Network Intrusion Detection". In: *Applied AI Letters* 7.2 (2026), e70026. DOI: 10.1002/ail2.70026. arXiv: 2410.07612 [cs.CR] (cit. on pp. 18, 33, 41, 42).